

Perimeter Defence in a Home Lab

Deploying pfSense, forwarding its logs into Wazuh, and writing the decoder that made them readable

Name: Muhammad Usman

Personal Project | August 2026



pfSense CE 2.7.2 | Wazuh 4.14.7 | Ubuntu 26.04 LTS | Kali Linux 2026.2 | Oracle VirtualBox 7.2

Contents

1. Project Overview	5
Purpose	5
Scope of the Work	5
What This Document Is.....	5
2. Lab Architecture and Log Pipeline	6
Machines and Roles	6
Network Design	6
Resource Allocation	7
The Log Pipeline.....	7
3. Design Decisions Taken Before Building	8
The Endpoint Moves Behind the Firewall.....	8
Both Interfaces Are Kept	8
The Block Rules Go on the Inner Interface	8
The Outer Address Is Fixed	8
Remote Logging Rather Than an Agent on the Firewall	8
4. Building the Firewall Machine	9
Creating the Virtual Machine.....	9
Attaching Two Networks	9
Installation	10
Assigning the Interfaces.....	11
Addressing	11
5. Moving the Endpoint Behind the Firewall	12
The New Position.....	12
What Had to Keep Working.....	13
6. Blocking Two Services by Alias.....	15
Aliases Rather Than Literal Addresses.....	15
The Rules.....	15
Placement	16
Verification	17
Reading the Counters	19
7. Geographic Filtering by Named Countries.....	19
Configuration	20
Choosing Countries Rather Than Regions	21
The Generated Rules	21

8. Domain Filtering	22
Configuration	22
Verification	23
9. Forwarding Firewall Logs to the Monitoring Platform	24
The Listener	24
Sending	24
Confirming Arrival.....	25
10. A Message the Platform Could Not Read	25
Reading the Failure.....	26
What Was Not Wrong.....	26
11. Writing a Decoder That Matches on Structure.....	27
The Idea	27
Four Attempts.....	27
Why the Patterns Look Like That.....	28
The Decoder.....	28
12. Detection Rules and Correlation.....	29
Three Rules	29
A Rule That Loaded and Would Not Fire	30
Validating Before Generating Traffic.....	31
The Live Alerts	32
Where the Alert Says It Came From	33
Correlation.....	34
Tracing a Rule End to End	35
13. Visualising the Firewall Activity	38
Saved Search.....	38
Visualisations	38
The Dashboard.....	39
14. Verification Summary and Limits of the Design	39
What Was Verified.....	39
Where the Design Cannot See	40
Known Imperfections in the Delivered Build.....	41
15. Problems Encountered	41
16. Key Takeaways.....	42
What I Would Do Differently	42
Appendix A: Command Reference.....	43

On the monitoring server	43
On the monitored endpoint	43
On the firewall console	44
Appendix B: Decoder and Rule Files	44
/var/ossec/etc/decoders/pfsense_custom_decoders.xml	44
/var/ossec/etc/rules/pfsense_custom_rules.xml	45
Appendix C: Sample Messages	45
A denial carrying no ports, as received.....	45
The same message decoded.....	46
A denial carrying ports, decoded.....	46

Part I: Design and Build

1. Project Overview

Purpose

Earlier this year I built a small monitoring laboratory: a Wazuh server watching a single Linux endpoint, with a custom decoder and rule written for an application log format the platform did not understand by default. That project ended at a boundary I could see clearly. A port scan run from the endpoint against the server produced no alert at all, because Wazuh reads logs written by the machines it monitors and a stealth scan never causes a service to write one. I noted at the time that closing that gap meant putting a device in the traffic path and feeding what it recorded into the platform.

This project does that. I added a pfSense firewall as a perimeter, moved the monitored endpoint behind it, blocked traffic at the firewall, and forwarded the firewall's logs into Wazuh so that a packet the firewall dropped becomes a classified, searchable security event rather than a line in a file nobody reads.

The interesting part turned out not to be the firewall. It was that the firewall and the monitoring platform disagreed about the shape of a log message. The decoder Wazuh ships for exactly this device could not match a single line of what arrived, and nothing was misconfigured on either side. Diagnosing that, and then writing a decoder that identifies these messages by their structure instead of by the field that was missing, is the core of the work and the reason this document exists.

Scope of the Work

- **Perimeter:** pfSense Community Edition installed as a third virtual machine with two interfaces, and the monitored endpoint moved onto the protected side of it.
- **Filtering:** two named services blocked by address alias, geographic filtering applied to named countries, and domain filtering attempted.
- **Transport:** firewall logs forwarded to the Wazuh manager over the system logging protocol, alongside the existing agent traffic rather than in place of it.
- **Detection engineering:** a custom decoder written because the one that ships with the platform could not match, and three rules including one that correlates repetition, mapped to MITRE ATT&CK.
- **Presentation:** a saved search, two visualisations and a dashboard built on the new detections.
- **Verification:** every capability claimed in this document tested and recorded in one table, including the two that did not hold.

What This Document Is

This is written so that someone else can repeat the build. Every configuration value used is stated, every command that mattered is included, and the problems are documented with the reasoning that identified each cause rather than only the final answer. Section 14 collects the verification results and the limits of

the design, Section 15 indexes every failure back to the section where it happened, and the appendices hold the commands, the two configuration files, and the raw messages the detections were written against.

2. Lab Architecture and Log Pipeline

Machines and Roles

Three machines run together on one laptop. The monitoring server and the endpoint already existed; the firewall is new, and the endpoint's position is the only part of the earlier environment this project changes.

Machine	Role	Attachment	Address	Memory
Ubuntu 26.04 LTS	Wazuh server, indexer and dashboard	NAT Network <code>NatNetwork</code>	10.0.2.20, fixed	6144 MB
pfSense 2.7.2	Firewall and router	<code>NatNetwork</code> on the outer side, Internal Network <code>LabLAN</code> on the inner	10.0.2.30 outer, 192.168.10.1 inner, both fixed	2048 MB
Kali Linux 2026.2	Monitored endpoint and traffic generator	<code>LabLAN</code>	192.168.10.100, assigned by the firewall	4096 MB
Windows host	Hypervisor, Oracle VirtualBox 7.2	Not on either laboratory network	n/a	n/a

Network Design

Two networks carry the laboratory, and the distinction between them is the whole basis of the design.

- **NAT Network `NatNetwork`, 10.0.2.0/24.** A shared segment. Every machine attached to it reaches the internet and can address every other machine on it directly. The monitoring server and the firewall's outer interface both sit here.
- **Internal Network `LabLAN`, 192.168.10.0/24.** Guest to guest only. Machines attached to it have no path to the host, no path to the internet, and no path to anything else, except through a router attached to both networks. That router is pfSense, and that is what makes it a perimeter rather than a filter standing beside the traffic.

The monitored endpoint has exactly one attachment, to `LabLAN`. Every packet it sends to anywhere outside 192.168.10.0/24 enters the firewall on the inner interface, is evaluated against the rules, and either leaves through the outer interface or is dropped and logged. There is no path around the firewall, and that property is what everything after Section 5 depends on.

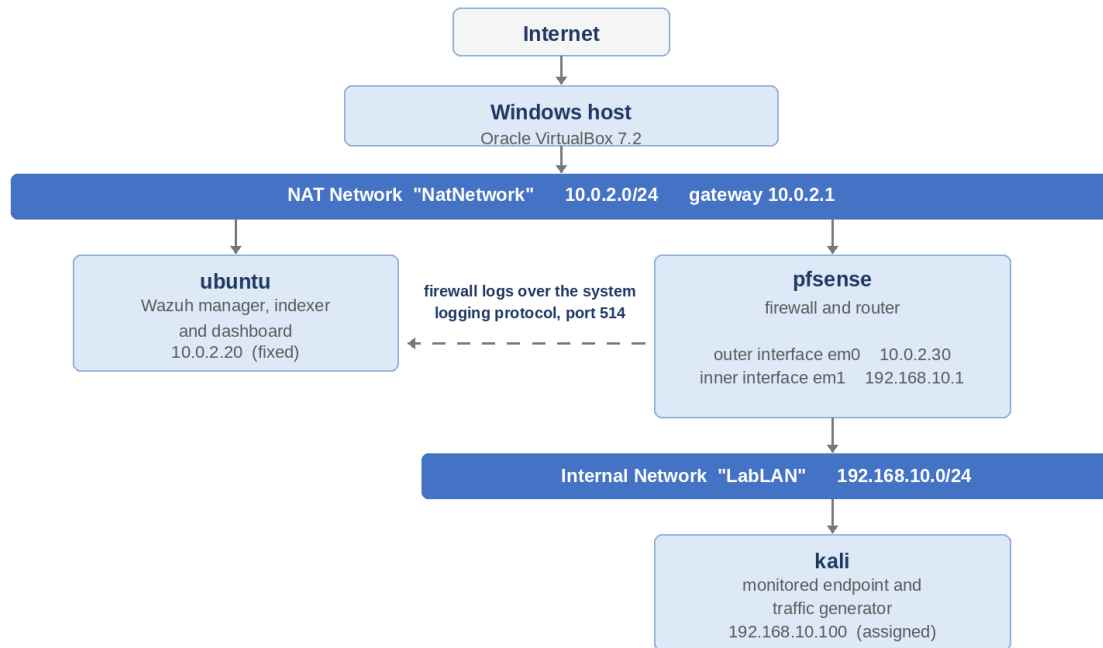


Figure 1: The two networks, the firewall bridging them, and the path firewall logs take back to the monitoring server

Resource Allocation

The firewall was given 2048 megabytes, which is comfortable for pfSense with the packages added in Sections 7 and 8. The monitoring server keeps the 6144 megabytes it was built with, because the indexer component reserves well over a gigabyte before any events arrive and under-allocating there does not produce an obvious error; it produces services that stop on their own and alerts that arrive minutes late. The endpoint keeps 4096 megabytes.

The Log Pipeline

Everything from Section 9 onward is a single pipeline with four stages. It is worth fixing the shape of it before the detail, because a fault in any one stage produces exactly the same symptom at the dashboard, which is that nothing appears.

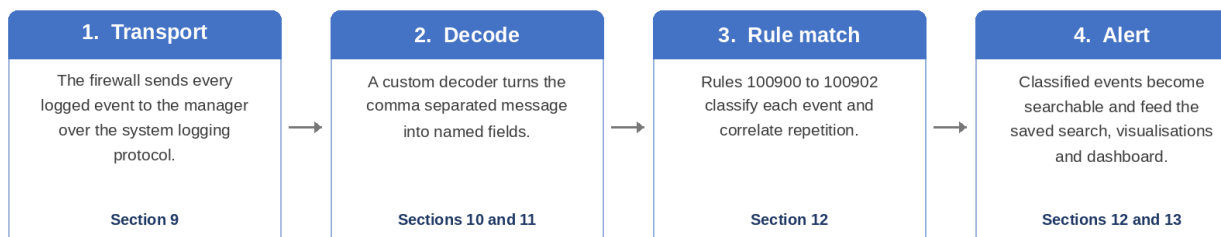


Figure 2: The four stages between a packet arriving at the firewall and an analyst seeing a classified alert, with the section covering each

3. Design Decisions Taken Before Building

Five decisions were made before the firewall was installed. Each avoids a specific failure, and each is the kind of thing that otherwise produces a configuration that reads correctly and does nothing at all.

The Endpoint Moves Behind the Firewall

A firewall can only act on traffic that passes through it. While the endpoint shared a network with the firewall's outer interface, its packets would go straight to the hypervisor's router at 10.0.2.1 and never reach pfSense. Every rule written later would read correctly, the packet counters would stay at zero, and the connections being blocked would succeed. Moving the endpoint onto the protected side is the only arrangement in which the rules mean anything.

The cost is one address. Everything built in the earlier project survives, because every mechanism in it runs outward from the agent to the manager rather than inward. Section 5 treats the move as a regression check and proves that rather than assuming it.

Both Interfaces Are Kept

pfSense can be run with a single interface, with the endpoint placed on the same segment. That produces something which filters, but it is not a perimeter: there is no inside and no outside, and the traffic it logs describes a shared network rather than a boundary being crossed. Keeping both interfaces creates a real boundary, and it is what makes the logs in Section 9 worth forwarding at all.

The Block Rules Go on the Inner Interface

pfSense evaluates rules on the interface where a packet **enters** the firewall. Traffic from the endpoint towards the internet enters on the inner interface, so that is where a rule denying it has to sit. Rules placed on the outer interface are evaluated against traffic arriving from outside and would never match outbound traffic from within.

This is easy to get wrong and the symptom is unhelpful, because the rule reads exactly as intended, the alias resolves, and the traffic passes anyway. The diagnostic that finds it is the packet counter, covered in Section 6.

The Outer Address Is Fixed

The manager accepts system log messages only from an address named explicitly in its configuration. If the firewall's outer address were assigned automatically and changed after a reboot, forwarding would stop and nothing would announce it. The alerts would simply cease, and the natural place to look would be the decoder, which would be the wrong place entirely. A fixed address at 10.0.2.30 removes the possibility.

Remote Logging Rather Than an Agent on the Firewall

pfSense is appliance software built on FreeBSD, intended to be managed through its own interface. Installing a monitoring agent onto it is neither supported nor sensible, and software added outside that

route survives upgrades badly. The device already speaks the system logging protocol natively, so the transport already exists and needs only to be pointed somewhere.

The trade-off is real and surfaces later. Messages arriving this way have no agent behind them, so the alerts they produce are attributed to the manager rather than to the machine that caused the activity. That is the subject of Section 12, and it is the single detail most likely to make an analyst conclude that a working detection is broken.

4. Building the Firewall Machine

Creating the Virtual Machine

The firewall runs as a third virtual machine. pfSense is built on FreeBSD, so the guest type is set accordingly rather than left at a Linux default, which changes the controller and network device defaults the hypervisor selects.

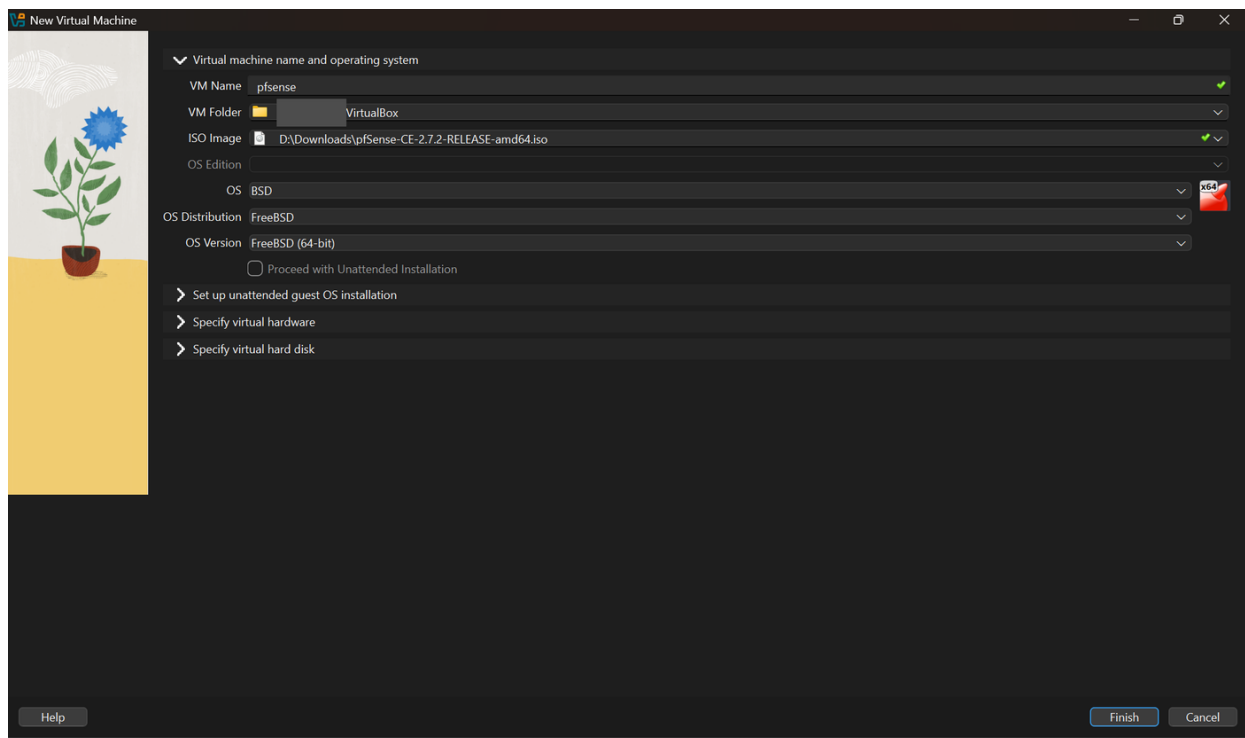


Figure 3: The firewall virtual machine defined with the installation image attached and the guest type set to FreeBSD

Attaching Two Networks

The two adapters are the machine's entire purpose. The first attaches to the shared NAT Network and becomes the outer interface. The second attaches to the internal network `LabLAN` and becomes the inner one. Both have to exist and be enabled before the installer runs, because the interface assignment step can only assign interfaces that are present.

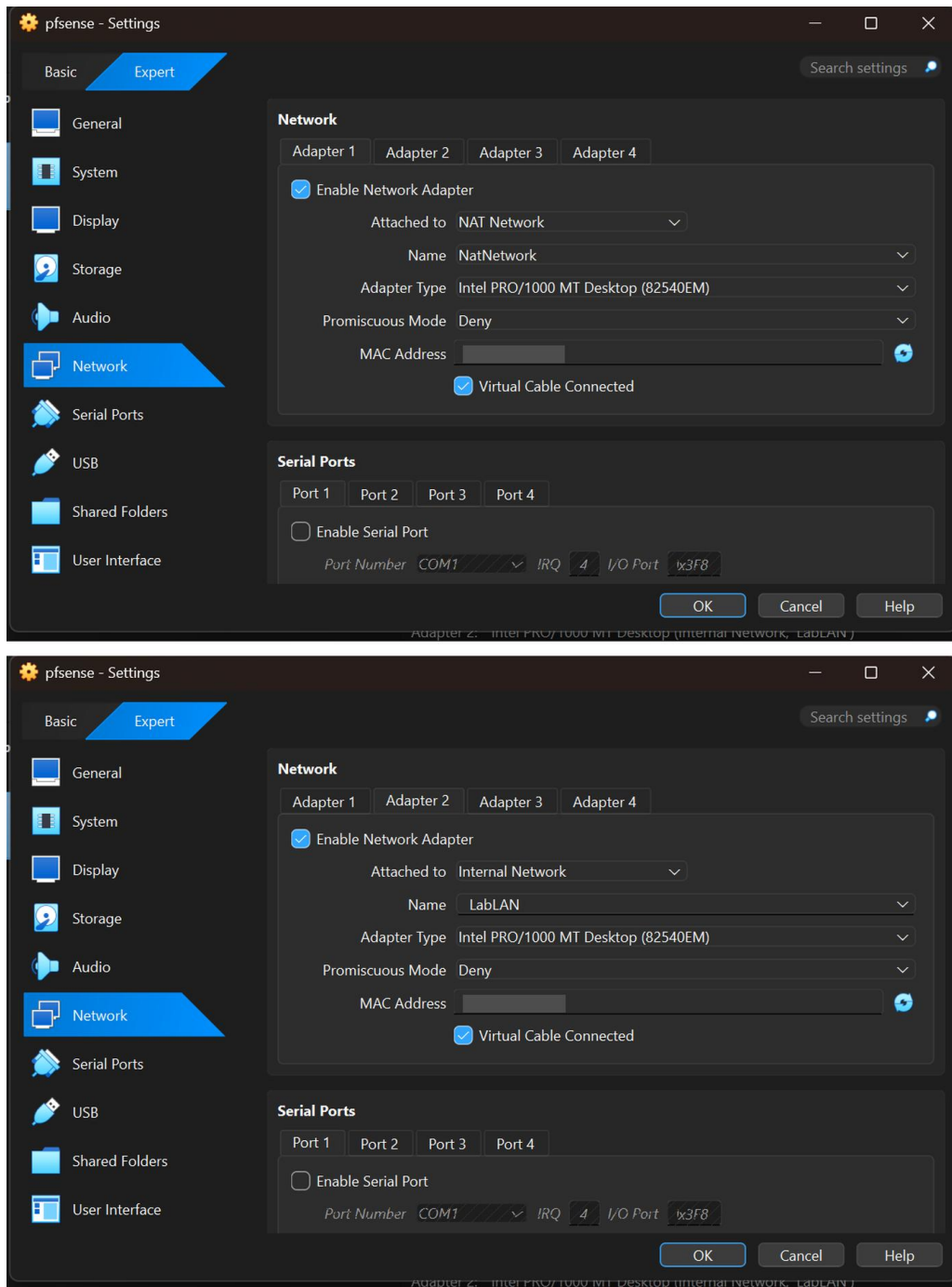


Figure 4: The first adapter on the shared NAT Network and the second on the internal network LabLAN

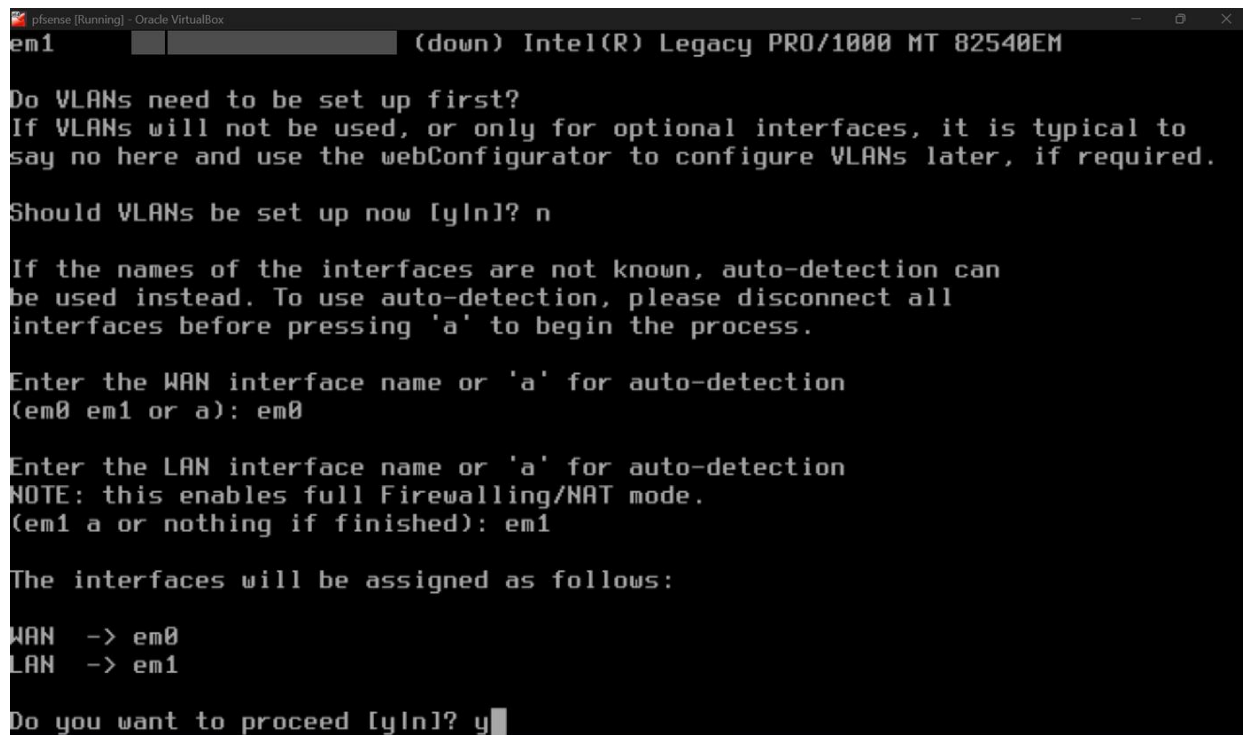
Installation

The installer runs with the guided defaults. I selected the single-device layout with no redundancy, which is the correct choice for one virtual disk: mirroring across devices that do not exist buys nothing.

One step is easy to miss. The installation image has to be detached from the virtual optical drive before the first real boot, or the machine boots into the installer again and it looks as though the installation failed.

Assigning the Interfaces

pfSense presents its two network devices as `em0` and `em1` and asks which is which. The order matters. `em0` corresponds to the first adapter, the one on the shared network, and becomes the interface labelled WAN; `em1` becomes the interface labelled LAN.

A screenshot of a pfSense console window running in Oracle VM VirtualBox. The window title is 'pfSense [Running] - Oracle VM VirtualBox'. The console output shows the following text:

```
em1 (down) Intel(R) Legacy PRO/1000 MT 82540EM

Do VLANs need to be set up first?
If VLANs will not be used, or only for optional interfaces, it is typical to
say no here and use the webConfigurator to configure VLANs later, if required.

Should VLANs be set up now [y/n]? n

If the names of the interfaces are not known, auto-detection can
be used instead. To use auto-detection, please disconnect all
interfaces before pressing 'a' to begin the process.

Enter the WAN interface name or 'a' for auto-detection
(em0 em1 or a): em0

Enter the LAN interface name or 'a' for auto-detection
NOTE: this enables full Firewalling/NAT mode.
(em1 a or nothing if finished): em1

The interfaces will be assigned as follows:

WAN -> em0
LAN -> em1

Do you want to proceed [y/n]? y
```

Figure 5: Interface assignment, with `em0` taking the outer role and `em1` the inner one

There is a trap in this prompt worth recording. At the local interface prompt the text reads `(em1 a or nothing if finished)`. Pressing Enter here to accept what looks like a default is read as "nothing", and the console answers that you have chosen to remove the local interface. The blank Enter belongs at the *Optional* prompt, one step later.

Addressing

The outer interface is set to `10.0.2.30/24`, and the inner interface to `192.168.10.1/24` with automatic assignment enabled for the range `192.168.10.100` to `192.168.10.200`.

I mistyped the outer address on the first attempt, entering `10.0.2.3` rather than `10.0.2.30`. That address sits inside the range the hypervisor hands out automatically and was the address the endpoint itself had been using, so the result would have been a conflict presenting as intermittent, inexplicable loss rather than a clean failure. Reading the console menu back after entering a value, instead of assuming the entry took, is what caught it.

```
pfSense [Running] - Oracle VirtualBox
The IPv4 WAN address has been set to 10.0.2.30/24
You can now access the webConfigurator by opening the following URL in your web
browser:
    https://10.0.2.30/
Press <ENTER> to continue.
VirtualBox Virtual Machine - Netgate Device ID:
*** Welcome to pfSense 2.7.2-RELEASE (amd64) on pfSense ***
WAN (wan)      -> em0      -> v4: 10.0.2.30/24
LAN (lan)      -> em1      -> v4: 192.168.10.1/24
0) Logout (SSH only)          9) pfTop
1) Assign Interfaces          10) Filter Logs
2) Set interface(s) IP address 11) Restart webConfigurator
3) Reset webConfigurator password 12) PHP shell + pfSense tools
4) Reset to factory defaults  13) Update from console
5) Reboot system              14) Enable Secure Shell (sshd)
6) Halt system                 15) Restore recent configuration
7) Ping host                   16) Restart PHP-FPM
8) Shell
Enter an option: 
```

Figure 6: Both interfaces addressed, with the outer side fixed at 10.0.2.30 and the inner side serving 192.168.10.0/24

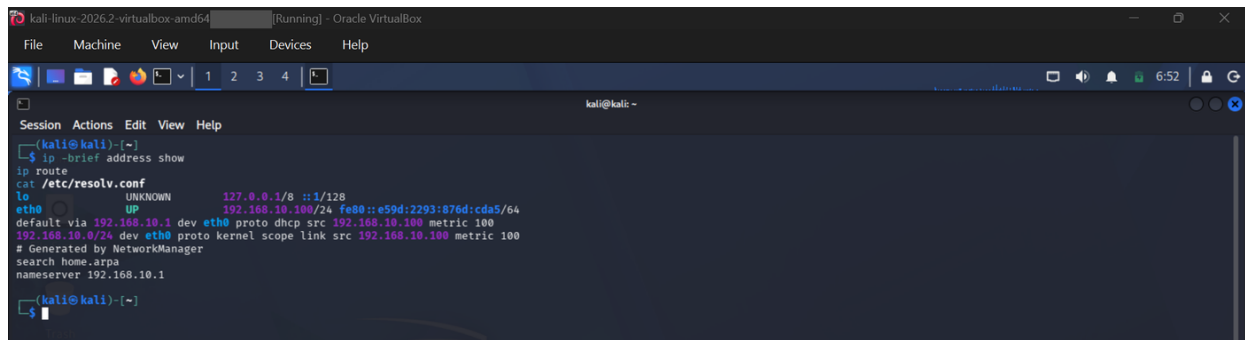
An address is not a route, and the two are separate objects. With both addresses set correctly, the firewall's own console could reach its gateway, the monitoring server and a public address, and yet nothing behind the firewall could reach anything at all. The cause was that no upstream gateway had been **selected** on the outer interface. The gateway object existed; it was simply not attached to the interface. The console test passing was actively misleading, because it exercises a path to a directly connected neighbour and proves nothing about the path that was broken. I spent time suspecting the rules first, which is the usual cause of that symptom.

5. Moving the Endpoint Behind the Firewall

Changing the endpoint's network attachment is the only structural change this project makes to the environment built earlier. Everything else is additive. Because it is the only change, it is worth treating as a regression check rather than as a configuration step: the question is not whether the endpoint gets an address, it is whether every mechanism built earlier still works from its new position.

The New Position

The endpoint's adapter moves from the shared network to the internal network `LabLAN`. It was carrying stale configuration from an earlier experiment, a fixed name server entry pointing at an address that no longer existed on any network, and that had to be cleared before it would accept configuration from the firewall at all.



```
kali@kali:~$ ip -brief address show
ip route
cat /etc/resolv.conf
to
eth0 UNKNOWN 127.0.0.1/8 ::1/128
default via 192.168.10.1 dev eth0 proto dhcp src 192.168.10.100 metric 100
192.168.10.0/24 dev eth0 proto kernel scope link src 192.168.10.100 metric 100
# Generated by NetworkManager
search home.arpa
nameserver 192.168.10.1
```

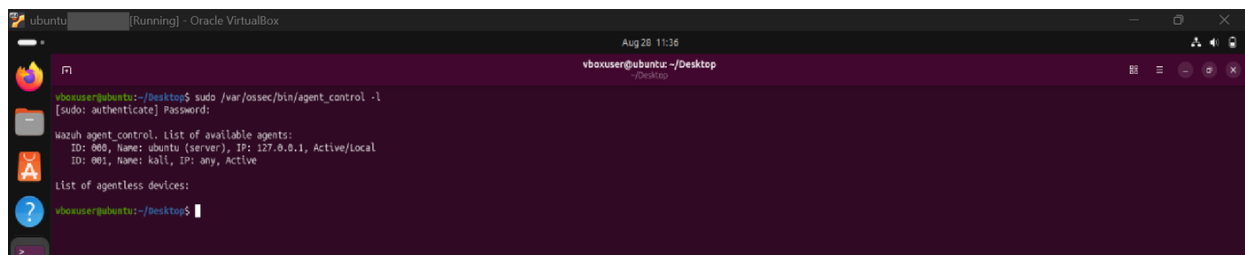
Figure 7: The endpoint's address, default route and name server after the move, all now pointing at the firewall

The address, the default route and the name server all resolve to 192.168.10.1, which is the firewall's inner interface. The search domain is the firewall's own.

What Had to Keep Working

Three things were built earlier and all three had to survive the move.

- **The agent.** It connects outward to the manager on port 1514, so it needs no reconfiguration; the connection simply now traverses the firewall.
- **Reachability of the firewall's own interface**, which is how everything in Sections 6 to 8 is configured.
- **Reachability of the monitoring dashboard**, which is how everything in Sections 12 and 13 is observed.



```
vboxuser@ubuntu:~/Desktop$ sudo /var/ossec/bin/agent_control -l
[sudo: authenticate] Password:
Wazuh agent_control. List of available agents:
ID: 000, Name: ubuntu (server), IP: 127.0.0.1, Active/Local
ID: 001, Name: kali, IP: any, Active
List of agentless devices:
vboxuser@ubuntu:~/Desktop$
```

Figure 8: The agent still reporting as active from its new position, with no configuration change made to it

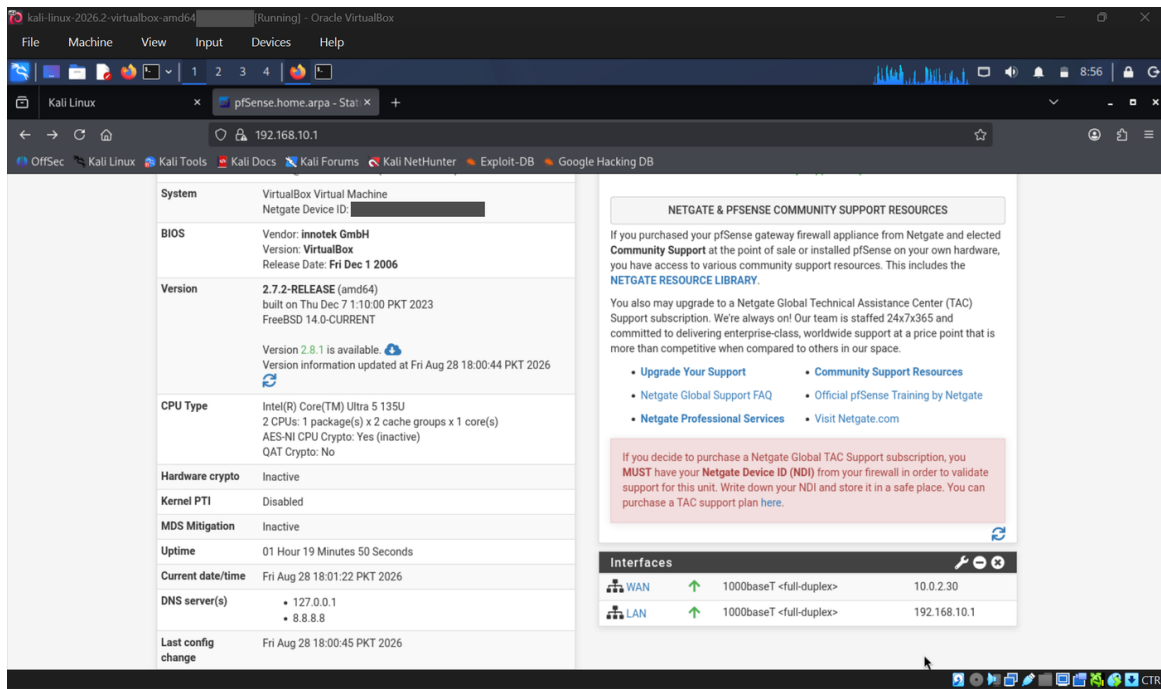


Figure 9: The firewall's management interface reached from the endpoint on the protected side

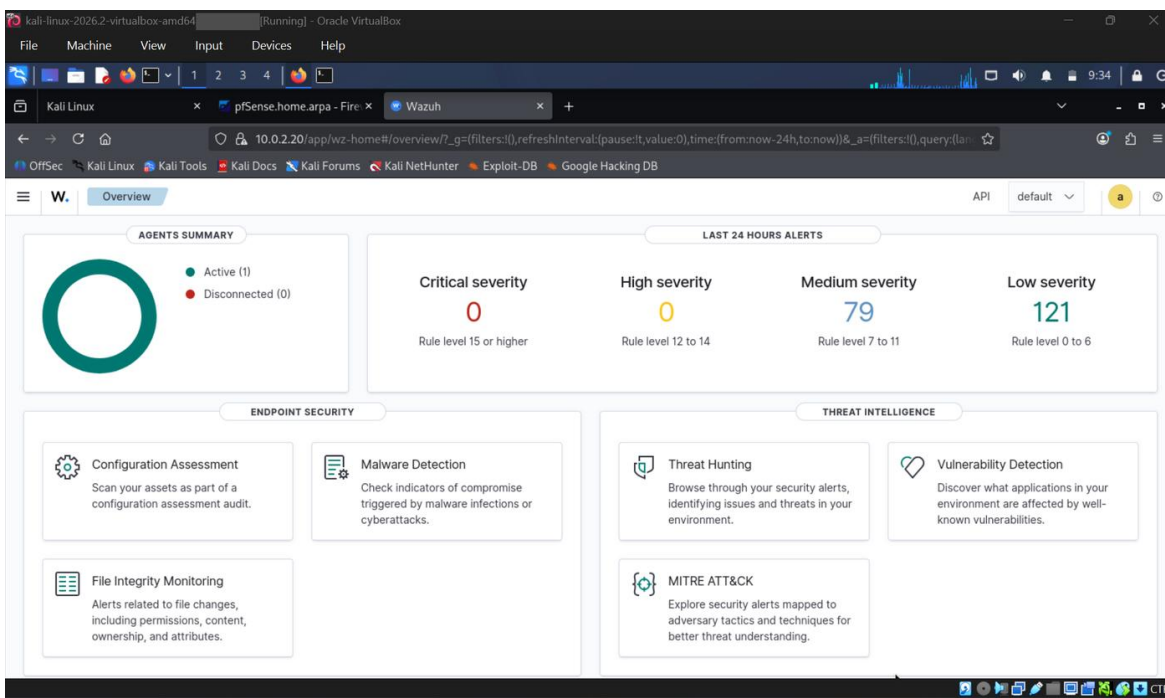


Figure 10: The monitoring dashboard still reachable from the endpoint, through the firewall

The agent reports as active with no configuration change at all, which is the point rather than a happy accident. Outbound mechanisms survive a change of network position. Inbound ones do not, and if the manager had been the party reaching out to the endpoint, this move would have broken the entire earlier build.

Part II: Filtering at the Perimeter

6. Blocking Two Services by Alias

Aliases Rather Than Literal Addresses

A rule can hold an address directly. Naming it first is better: the rule then reads as intent rather than as a number, and when the address changes there is one place to change it rather than every rule that referred to it. I created one host alias per service.

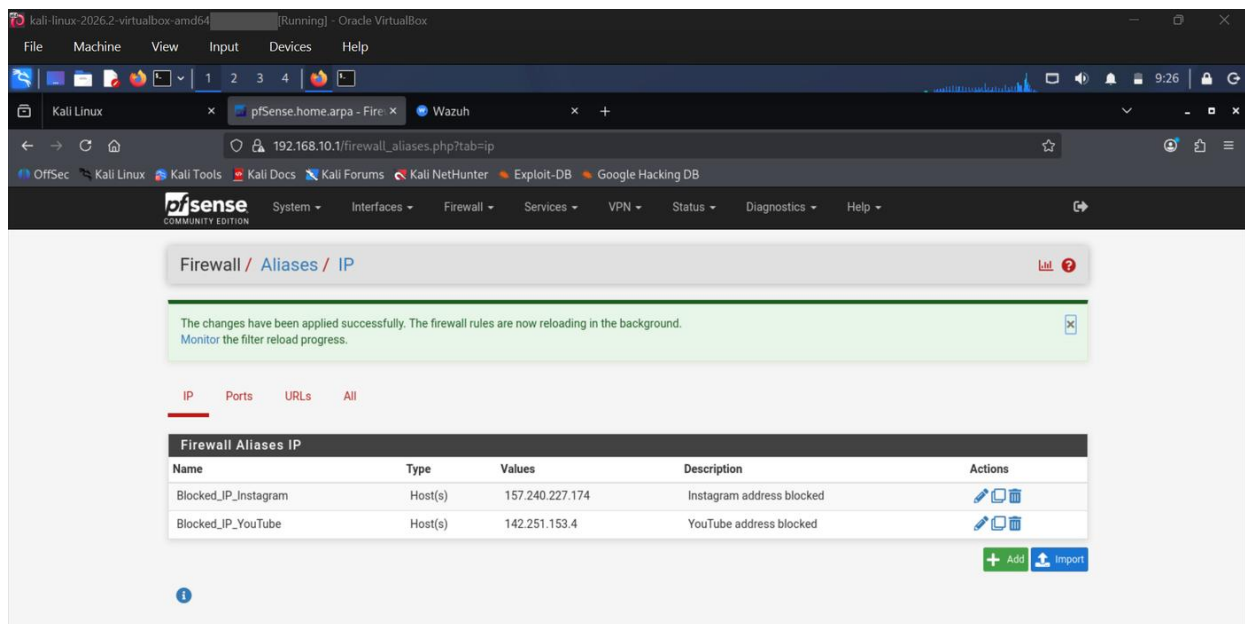


Figure 11: The two host aliases, each holding a single address with a description of what it represents

Both services answer from large and constantly changing address pools, so the two addresses here are the ones that resolved on the day the rules were written rather than permanent facts about either service. Anyone repeating this should resolve their own and use what comes back.

The Rules

Interface	Action	Source	Destination	Description
Inner (LAN)	Block	192.168.10.100	Blocked_IP_YouTube	Block YouTube address from Kali
Inner (LAN)	Block	192.168.10.100	Blocked_IP_Instagram	Block Instagram address from Kali

Both rules have logging enabled, which is what makes them visible to everything from Section 9 onward. A rule that blocks without logging protects the network and tells the monitoring platform nothing.

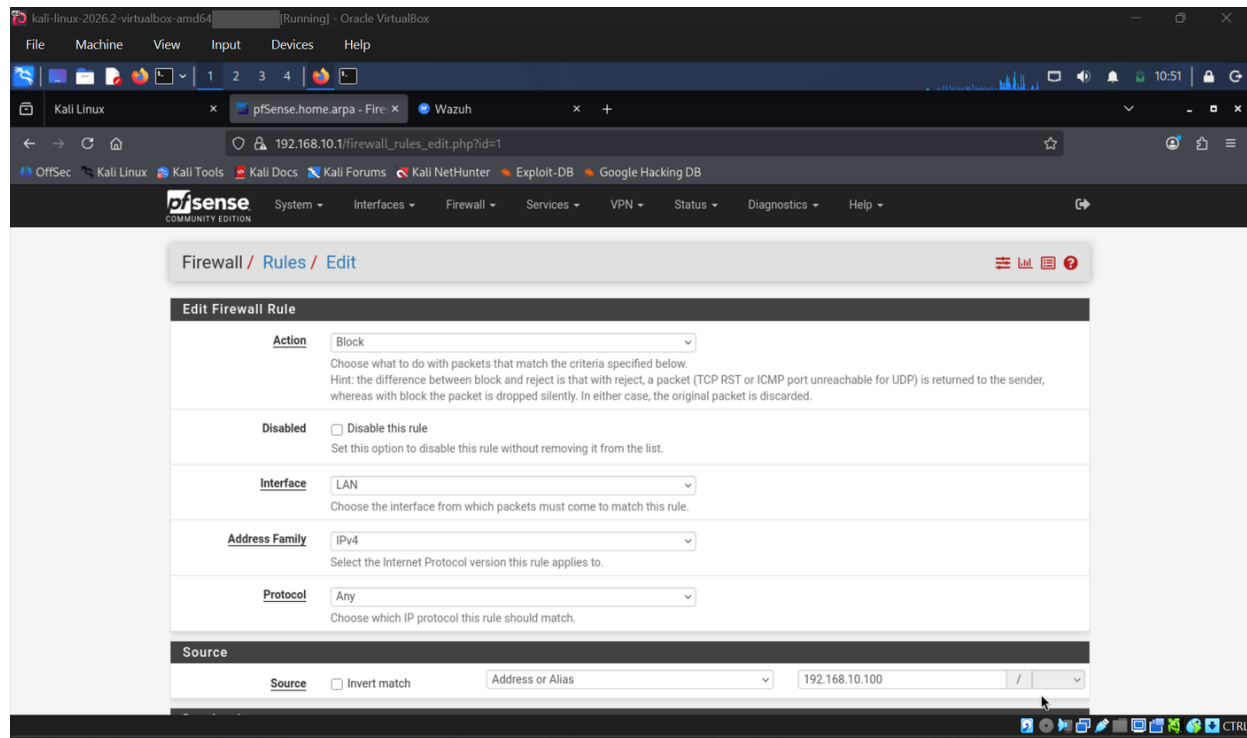


Figure 12: The first block rule, set to deny traffic entering on the inner interface from the endpoint's address

Placement

Two properties decide whether these rules do anything at all. They sit on the inner interface, for the reason given in Section 3, and they sit **above** the default rule that permits everything from the local network outward. pfSense evaluates rules from the top and stops at the first match, so a block rule positioned below a permit rule that matches everything is never reached.

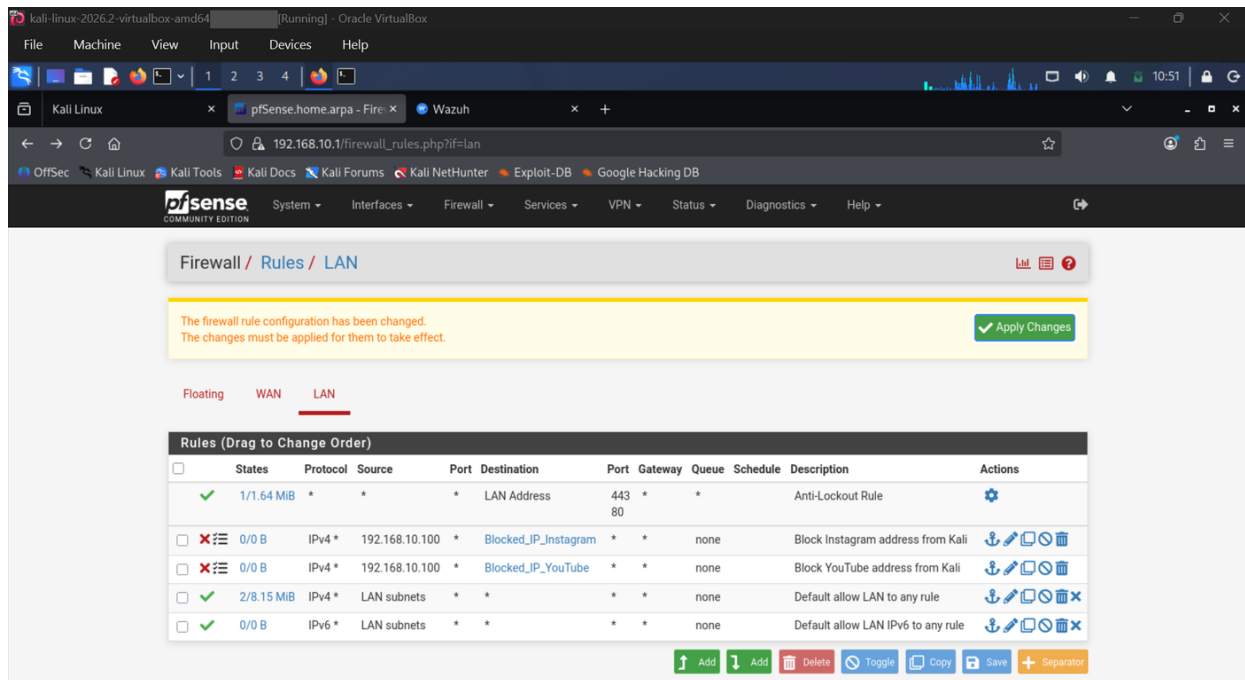
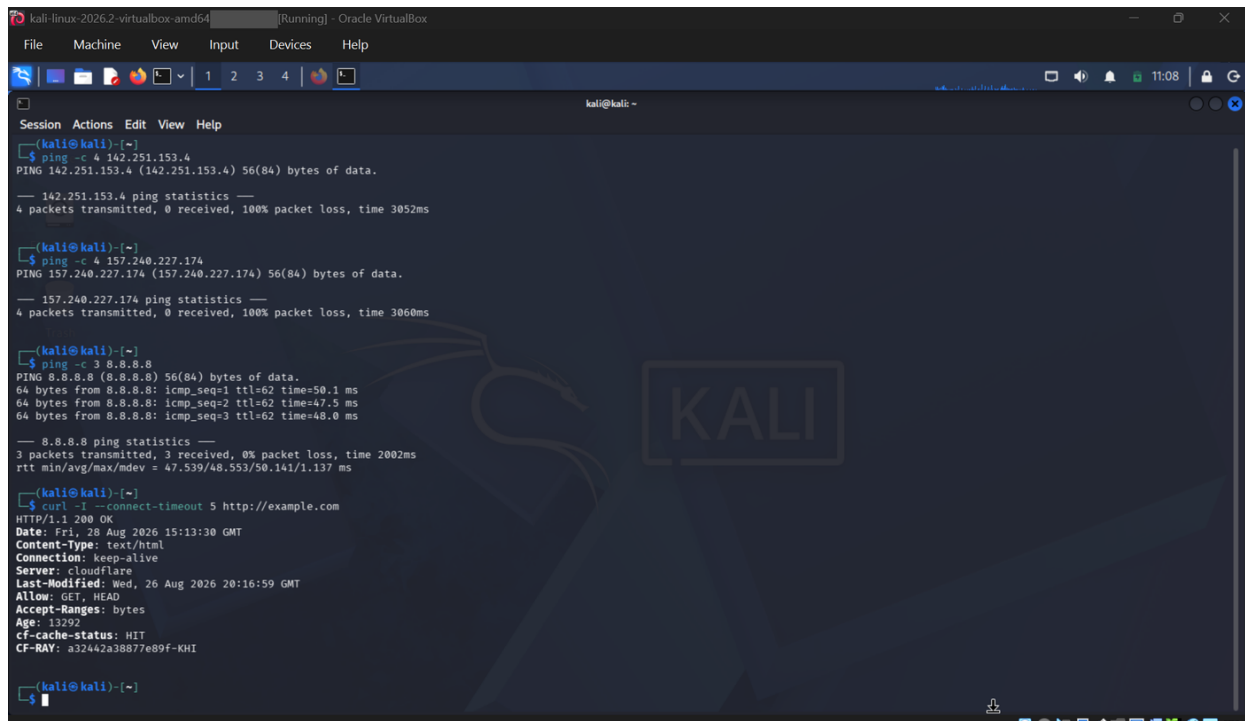


Figure 13: The completed rule list on the inner interface, with both block rules above the default permit rule

Verification

A rule that has been saved is not a rule that works. The test is to attempt the blocked destinations from the endpoint and confirm two things at once: that they fail, and that unrelated destinations still succeed. The second half matters as much as the first, because a rule that blocks everything also blocks these two addresses, and only the control proves the rule is doing what it claims.



```
kali@kali:~$ ping -c 4 142.251.153.4
PING 142.251.153.4 (142.251.153.4) 56(84) bytes of data.
--- 142.251.153.4 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3052ms

kali@kali:~$ ping -c 4 157.240.227.174
PING 157.240.227.174 (157.240.227.174) 56(84) bytes of data.
--- 157.240.227.174 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3060ms

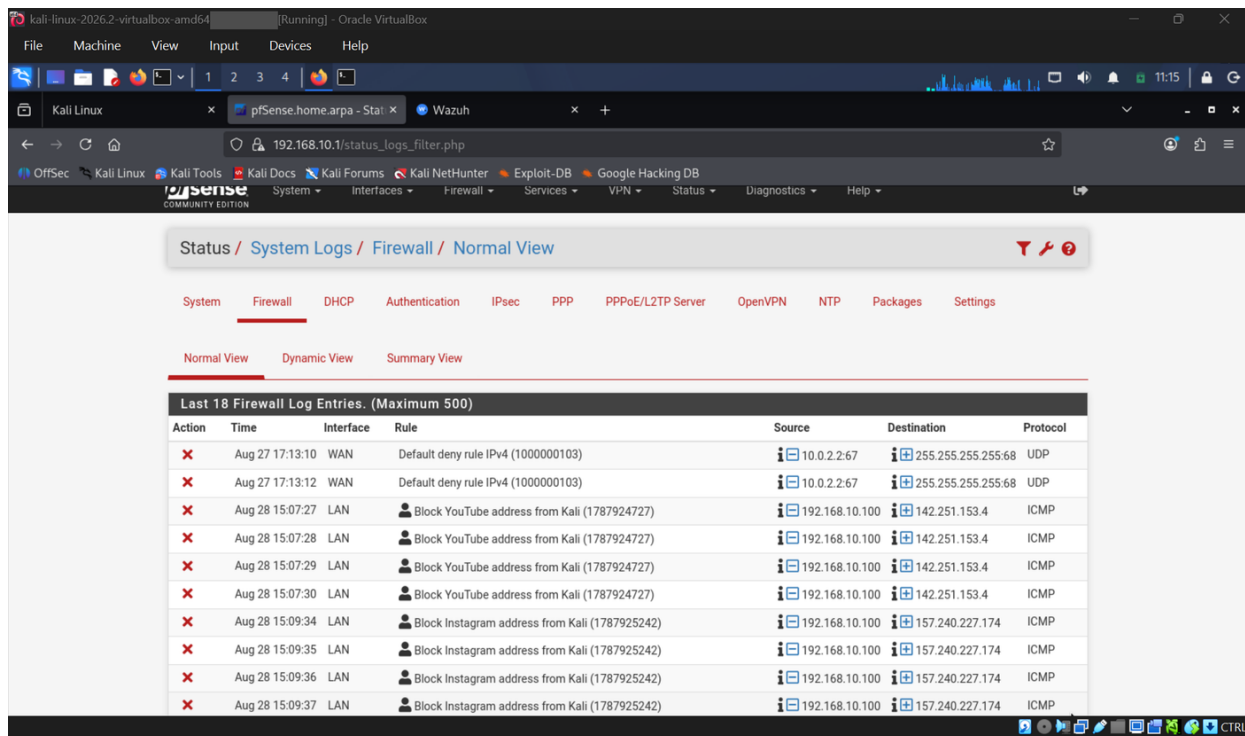
kali@kali:~$ ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=62 time=50.1 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=62 time=47.5 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=62 time=48.0 ms
--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 47.539/48.553/50.141/1.137 ms

kali@kali:~$ curl -i --connect-timeout 5 http://example.com
HTTP/1.1 200 OK
Date: Fri, 28 Aug 2026 15:13:30 GMT
Content-Type: text/html
Connection: keep-alive
Server: cloudflare
Last-Modified: Wed, 26 Aug 2026 20:16:59 GMT
Allow: GET, HEAD
Accept-Ranges: bytes
Age: 13292
cf-cache-status: HIT
CF-RAY: a32442a38877e89f-KHI

kali@kali:~$
```

Figure 14: Total loss to both blocked addresses, with a public address and an unrelated web server answering normally in the same session

The firewall's own log records each denial with the description of the rule that caused it, which is what makes a descriptive rule name worth the effort of writing.



Action	Time	Interface	Rule	Source	Destination	Protocol
✗	Aug 27 17:13:10	WAN	Default deny rule IPv4 (1000000103)	10.0.2.2:67	255.255.255.255:68	UDP
✗	Aug 27 17:13:12	WAN	Default deny rule IPv4 (1000000103)	10.0.2.2:67	255.255.255.255:68	UDP
✗	Aug 28 15:07:27	LAN	Block YouTube address from Kali (1787924727)	192.168.10.100	142.251.153.4	ICMP
✗	Aug 28 15:07:28	LAN	Block YouTube address from Kali (1787924727)	192.168.10.100	142.251.153.4	ICMP
✗	Aug 28 15:07:29	LAN	Block YouTube address from Kali (1787924727)	192.168.10.100	142.251.153.4	ICMP
✗	Aug 28 15:07:30	LAN	Block YouTube address from Kali (1787924727)	192.168.10.100	142.251.153.4	ICMP
✗	Aug 28 15:09:34	LAN	Block Instagram address from Kali (1787925242)	192.168.10.100	157.240.227.174	ICMP
✗	Aug 28 15:09:35	LAN	Block Instagram address from Kali (1787925242)	192.168.10.100	157.240.227.174	ICMP
✗	Aug 28 15:09:36	LAN	Block Instagram address from Kali (1787925242)	192.168.10.100	157.240.227.174	ICMP
✗	Aug 28 15:09:37	LAN	Block Instagram address from Kali (1787925242)	192.168.10.100	157.240.227.174	ICMP

Figure 15: The firewall log recording the denials, with the rule description, source and destination all visible

Reading the Counters

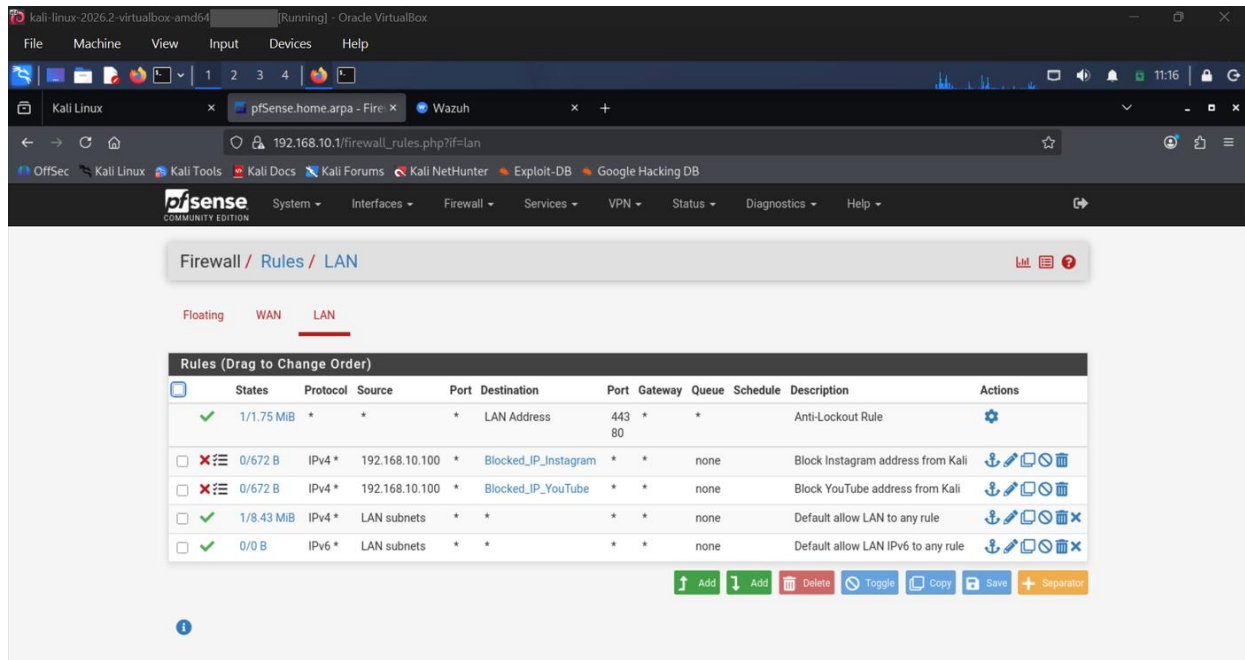


Figure 16: Both block rules matching traffic, with the byte counters advanced from zero

The counters moved from nothing to 672 bytes on both rules once traffic had been attempted. The first figure in that column is the state count, and it stays at zero permanently. This looks like a failure and is not: a blocked packet never establishes a connection state, so there is nothing to count. The byte figure is the one that says the rule is matching. A reading of 0/0 B is the real failure signal, and it is exactly what these rules would show had they been placed on the outer interface.

7. Geographic Filtering by Named Countries

The two rules in Section 6 deny two addresses. Geographic filtering is the same idea applied to tens of thousands of addresses at once, using a database that maps address ranges to countries. I installed the pfBlockerNG package for this.

Configuration

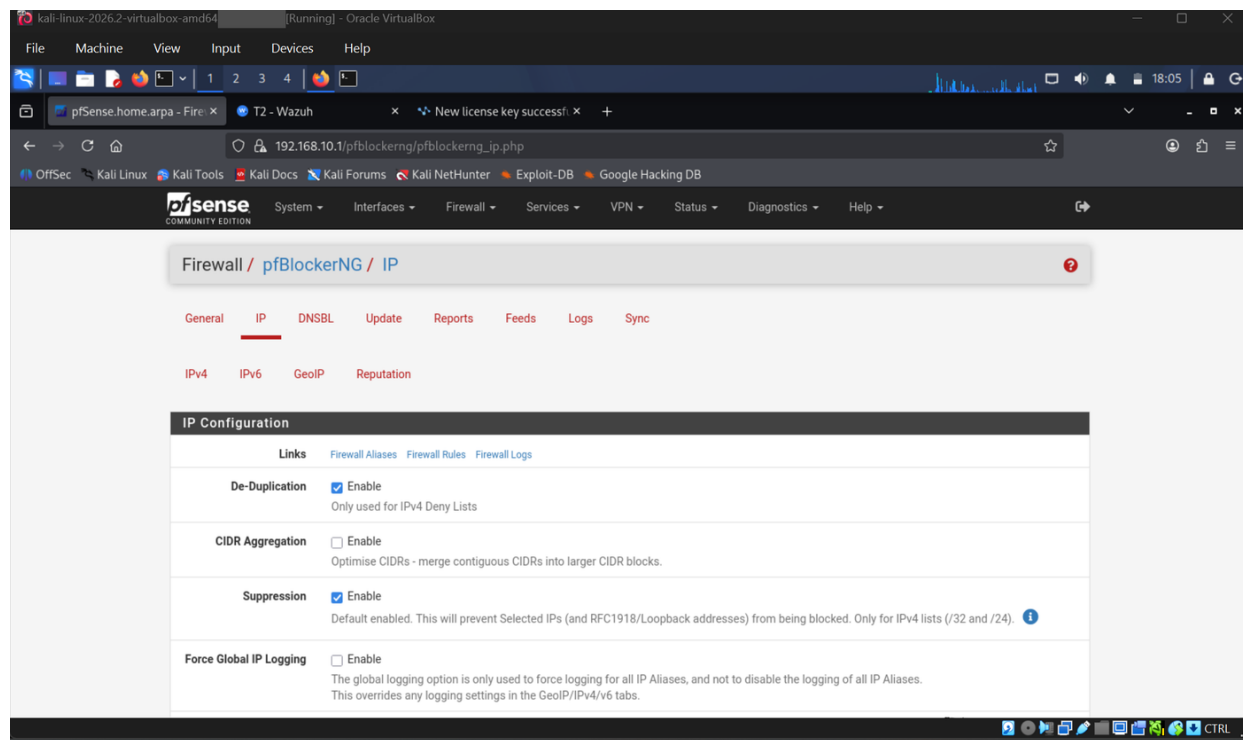


Figure 17: The address filtering configuration, with de-duplication and suppression of private ranges enabled

The package needs an account with the geographic database provider and a licence key, entered once and then used to fetch updates automatically. Both are obscured in the figure below for the obvious reason: a licence key is a credential, and a credential visible in a screenshot in a document that will be shared is a credential that has been published.

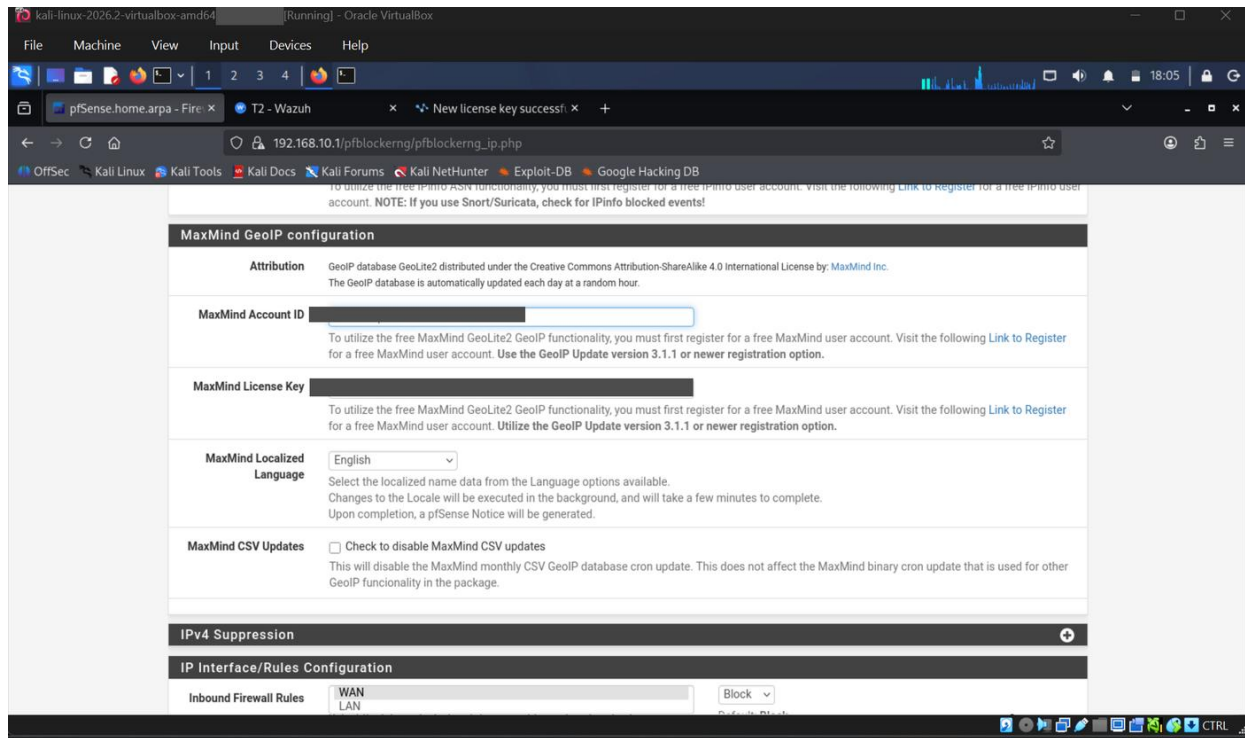


Figure 18: The geographic database account configured, with the account identifier and licence key obscured

Choosing Countries Rather Than Regions

The interface makes it easy to select an entire continent with a single click, and that is the wrong control to reach for here. This laboratory sits in Pakistan. Denying Asia as a region would have denied the country the laboratory itself runs in, and the first symptom would have been the whole environment losing connectivity with the cause several menus away from where anyone would start looking.

I selected individual countries instead: China and India within Asia, and France and Germany within Europe, with logging enabled and traffic denied in both directions. The resulting alias tables held 13,251 entries for the Asian selection and 29,878 for the European one.

The general point is worth stating plainly, because it is not really about this package. A blunt control applied at a boundary hits everything on the far side of it, including the things nobody pictured when they clicked. Naming what you mean is slower and produces a filter that can still be reasoned about a month later.

The Generated Rules

pfBlockerNG does not ask the administrator to write rules for these lists. It builds the alias tables and generates its own rules referring to them, which then appear alongside the hand-written ones and are evaluated in the same order.

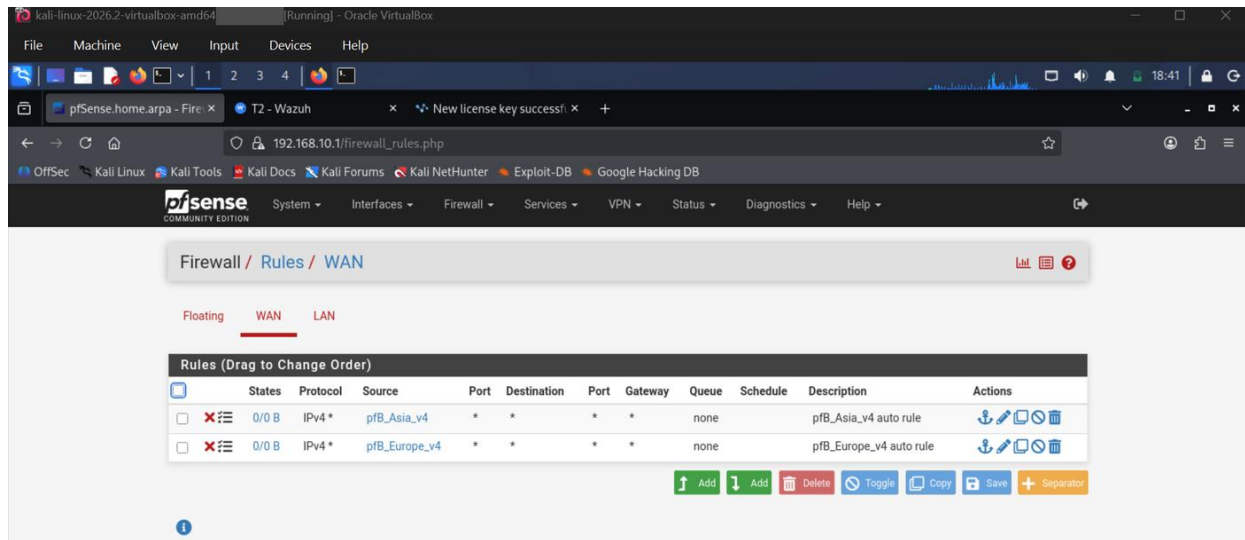


Figure 19: Rules generated automatically by the package, referring to the country alias tables it maintains

One property of this package is worth knowing before troubleshooting it. Settings take effect on a reload run, not on save. A group can be saved correctly and do nothing whatsoever until a reload has completed, which makes it very easy to conclude that a correct configuration is broken and start changing it.

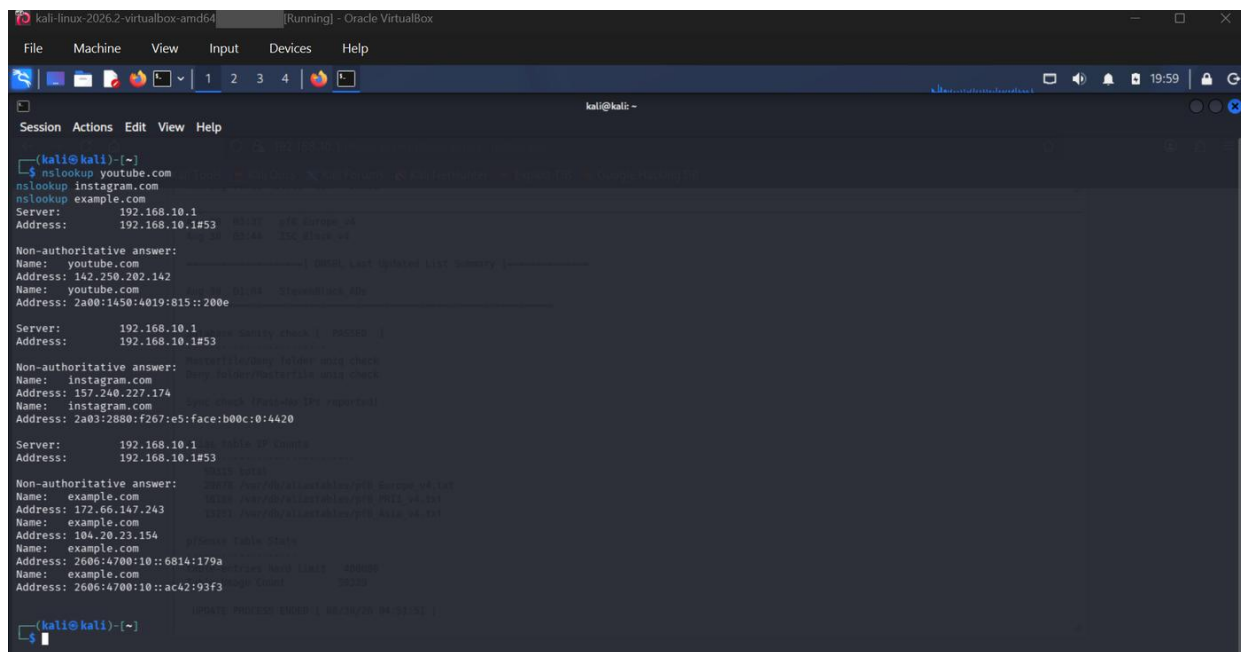
8. Domain Filtering

Address filtering blocks a number. Domain filtering blocks a name, by having the firewall's own resolver answer with a sinkhole address instead of the real one. In principle it is the more useful of the two, because names are stable while addresses drift. In this build it is the part that did not work.

Configuration

The mechanism needs somewhere to send the traffic it intercepts, so I created a virtual address of 10.10.10.1/32 on the loopback interface for the small web server the package uses for that purpose, then created a block group containing the two service domains with the resolver-based action. Two identical virtual addresses had been created by mistake and one had to be removed before the configuration would settle.

Verification



```
kali@kali:~$ nslookup youtube.com
Server:      192.168.10.1
Address:     192.168.10.1#53
Non-authoritative answer:
Name:   youtube.com
Address: 142.250.202.142
Name:   youtube.com
Address: 2a00:1450:4019:815::200e

kali@kali:~$ nslookup instagram.com
Server:      192.168.10.1
Address:     192.168.10.1#53
Non-authoritative answer:
Name:   instagram.com
Address: 157.240.227.174
Name:   instagram.com
Address: 2a03:2880:f267:e5:face:b00c:0:4420

kali@kali:~$ nslookup example.com
Server:      192.168.10.1
Address:     192.168.10.1#53
Non-authoritative answer:
Name:   example.com
Address: 172.66.147.243
Name:   example.com
Address: 104.20.23.154
Name:   example.com
Address: 2606:4700:10::6814:179a
Name:   example.com
Address: 2606:4700:10::ac42:93f3
```

Figure 20: Name lookups from the endpoint returning the real public addresses for both listed domains rather than the sinkhole address

Neither domain was sinkholed. Lookups for an unrelated control domain behaved normally in the same session, so the resolver itself was working; it was simply answering these two names truthfully.

I am reporting this as unresolved rather than as working, because it is unresolved. Two causes are known to produce exactly this symptom, and neither could be confirmed or excluded here.

- **The sinkhole address not being present.** If the virtual address is missing from the loopback interface, the package disables domain blocking and records the reason in its own update log rather than anywhere in the interface, so the configuration continues to look correct while doing nothing.
- **Filtered search resolution.** Enabling the safe-search option makes this worse rather than better for one of these two domains, because that option works by resolving the domain to a filtered address. It guarantees the name is answered, which is precisely the opposite of sinkholing it.

There is also a limit that applies even when domain filtering is configured perfectly, and it is significant enough that it belongs in Section 14 rather than here.

Part III: Teaching the Platform to Read the Firewall

9. Forwarding Firewall Logs to the Monitoring Platform

The Listener

The manager already listened for agents on port 1514. It now needs a second listener for messages arriving over the system logging protocol, which is a different transport with no agent behind it. The block below was added to `/var/ossec/etc/ossec.conf` **after** the existing agent listener rather than in place of it.

```
<remote>
  <connection>syslog</connection>
  <port>514</port>
  <protocol>udp</protocol>
  <allowed-ips>10.0.2.30</allowed-ips>
  <local_ip>10.0.2.20</local_ip>
</remote>
```

Every element there is load-bearing. `<allowed-ips>` restricts acceptance to the firewall's outer address, so the manager will not ingest messages from anything else that happens to be on the segment. `<local_ip>` pins the listener to one interface address rather than to every address on the machine. And the position of the block matters more than it appears: replacing the existing agent listener instead of adding after it would disconnect the endpoint's agent, and the symptom would be the endpoint falling silent at the same moment firewall logging was switched on, which invites entirely the wrong diagnosis.

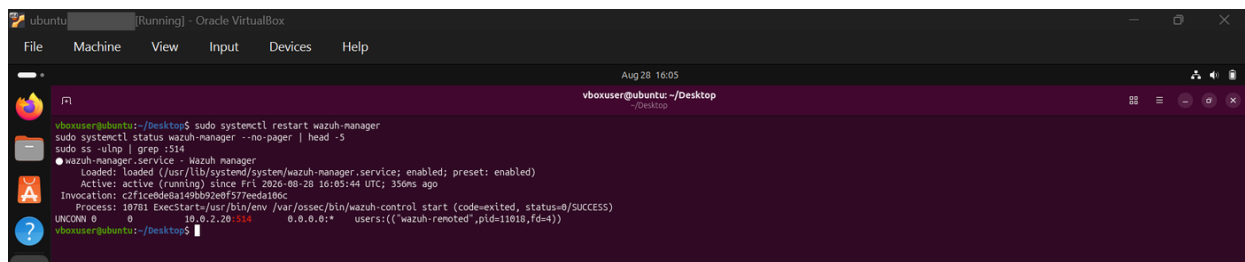


Figure 21: The system log listener confirmed bound, with wazuh-remoted holding port 514 on the manager's own address

The listener is confirmed bound before anything is sent to it. If the receiving half is not there, no amount of configuration on the firewall will produce a result.

Sending

On the firewall, remote logging is enabled with the manager as the destination, the source address pinned to the outer interface so that the sending address matches what `<allowed-ips>` expects, and two settings that decide whether any of this works at all.

- **Message format.** Set to the older of the two standards. The newer structured format is not what the manager's pre-decoding stage expects, and selecting it produces messages that arrive and are then discarded in silence. This is the most common cause of a transport that is provably working and produces no alerts.

- **Categories.** Only system events and firewall events are forwarded, out of the dozen or so the firewall can send. This is a detection decision rather than a convenience: the rules written in Section 12 act on firewall messages, and every other category is noise that dilutes the event stream, buries the events that matter and makes the dashboard in Section 13 harder to read rather than richer.

Confirming Arrival

Before touching a decoder it is worth proving that the messages are physically arriving, because the alternative is debugging a decoder against messages that were never delivered.

```
sudo tcpdump -n -i enp0s3 -A udp port 514
```

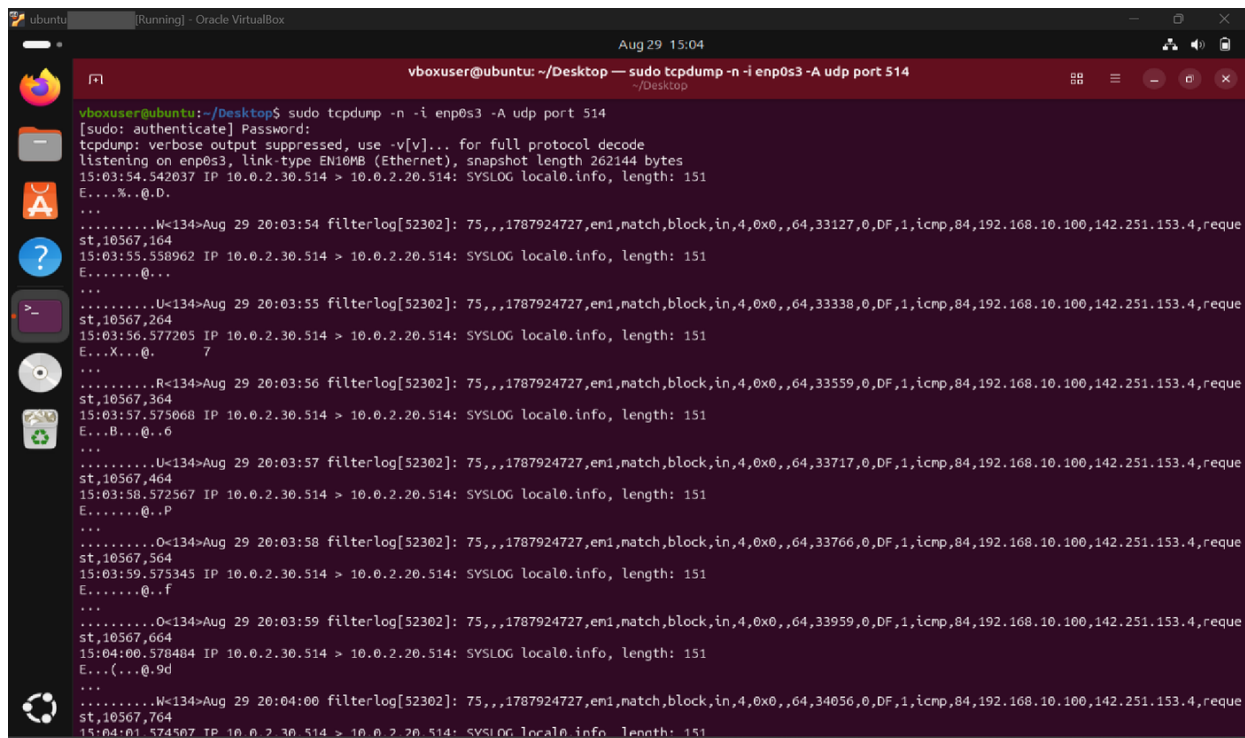


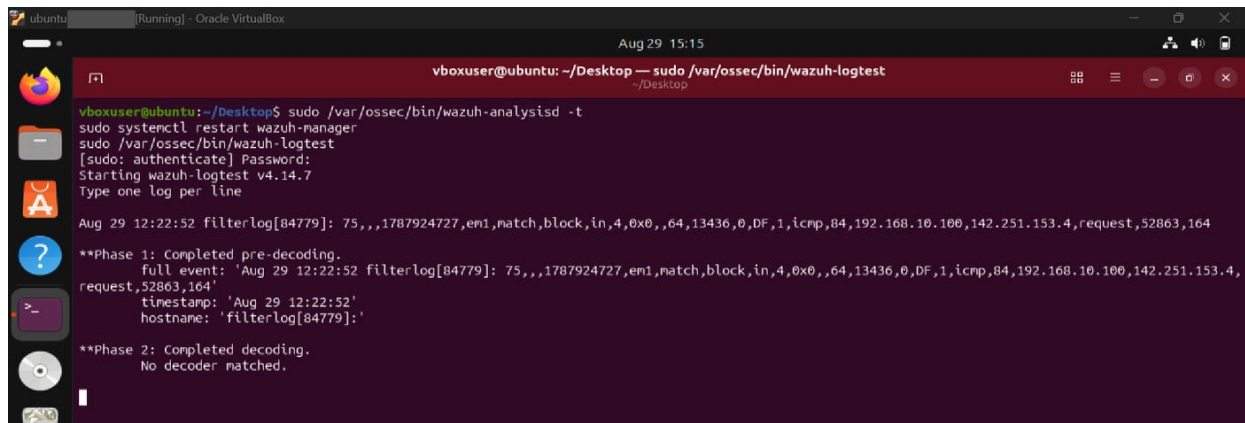
Figure 22: Firewall messages captured arriving at the manager on port 514, each one a single comma separated line

They arrive. Each message is one line carrying the interface, the action, the direction, the protocol, both addresses and both ports. The transport works. Everything from here is about whether the platform can read what it has been handed.

10. A Message the Platform Could Not Read

Wazuh ships a decoder for pfSense. The device is common enough that support for it is built in, and the reasonable expectation at this point was that the messages now arriving would be decoded and classified with no further work.

They were not decoded at all.



```
vboxuser@ubuntu: ~/Desktop$ sudo /var/ossec/bin/wazuh-logtest -t
sudo systemctl restart wazuh-manager
sudo /var/ossec/bin/wazuh-logtest
[sudo: authenticate] Password:
Starting wazuh-logtest v4.14.7
Type one log per line

Aug 29 12:22:52 filterlog[84779]: 75,,,1787924727,en1,match,block,in,4,0x0,,64,13436,0,DF,1,icmp,84,192.168.10.100,142.251.153.4,request,52863,164

**Phase 1: Completed pre-decoding.
full event: 'Aug 29 12:22:52 filterlog[84779]: 75,,,1787924727,en1,match,block,in,4,0x0,,64,13436,0,DF,1,icmp,84,192.168.10.100,142.251.153.4,request,52863,164'
timestamp: 'Aug 29 12:22:52'
hostname: 'filterlog[84779]:'

**Phase 2: Completed decoding.
No decoder matched.
```

Figure 23: The validation tool reporting that no decoder matched, with the program name sitting in the hostname field

Reading the Failure

The validation tool reports its work in phases, and the failure is visible in the first phase rather than the second.

A message in this format is expected to carry, in order, a priority marker, a timestamp, a **hostname**, a program name, and then the message body. The pre-decoding stage reads those fields positionally: first field the timestamp, second the hostname, third the program name.

pfSense sent no hostname. The field is simply absent, and the message runs straight from the timestamp to the program name. Reading positionally, the pre-decoder therefore recorded `filterlog[84779]:` as the *hostname*, and found no program name at all.

That single displacement is enough to defeat the decoder. The decoder that ships with the platform identifies these messages **by program name**, and the program name it looks for is now sitting in a different field under a different label. It cannot match, it does not match, and the second phase reports `No decoder matched` for a message that arrived perfectly and was formatted exactly as the firewall intended.

What Was Not Wrong

It is worth being precise about this, because the instinct on seeing that output is to go looking for a mistake. The firewall was configured correctly and was sending in the format it had been asked to send in. The manager was configured correctly and was receiving. The decoder was present, was loaded, and was the right decoder for the device. Nothing was broken.

Two products simply disagreed about whether an optional field is optional.

I tried the obvious remedies first: setting the system hostname explicitly, re-saving the logging configuration, and finally rebooting the firewall. The field never appeared. At that point the choice is to abandon either the shipped decoder or the integration, and only one of those is a problem with an answer.

11. Writing a Decoder That Matches on Structure

The Idea

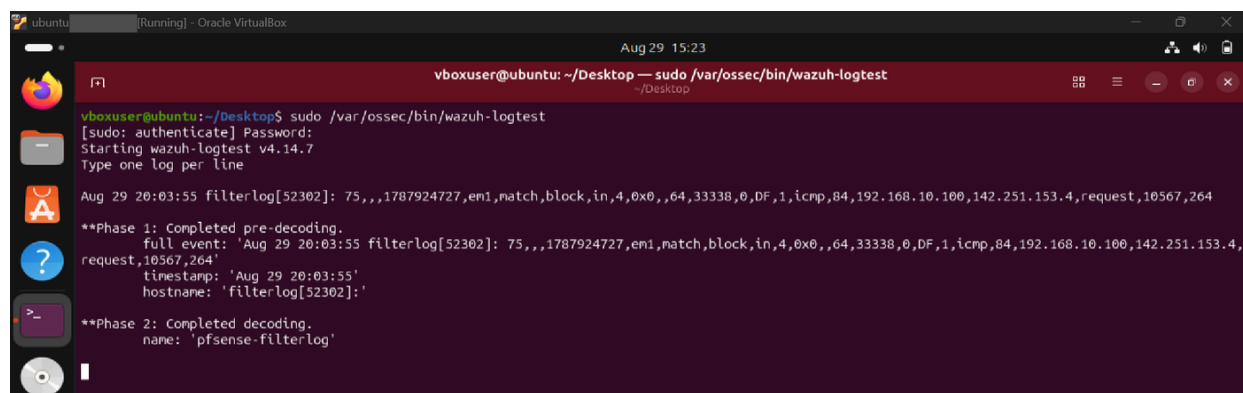
If a message cannot be identified by a field that is missing, it has to be identified by something that is present. Every one of these firewall messages contains the literal text `,match,` at a fixed position in its comma separated body, and nothing else the firewall sends contains it. That string is a reliable fingerprint for this message type, and it is independent of hostnames, program names and anything else the pre-decoder may have shuffled.

So the decoder identifies the message by its structure and extracts fields by counting commas from that anchor, rather than by naming the program that produced it.

Four Attempts

The working decoder took four attempts, and the three failures are more instructive than the result.

1. **Anchor the pre-match at the start of the line** and count fields from the beginning. No match at all. The pre-decoder has already consumed part of the line, so the start of the string is not where I assumed it was.
2. **Reduce the pre-match to the literal `,match,`**. The parent decoder matched, which was the first real progress, but no fields were extracted from it.
3. **Add an offset so the child patterns begin after the parent match**. No change whatsoever.
4. **Re-anchor the child patterns on `, (\S+) ,match,`** rather than counting from the start of the string. Fields extracted correctly.



```
vboxuser@ubuntu: ~/Desktop — sudo /var/ossec/bin/wazuh-logtest
~/Desktop
vboxuser@ubuntu:~/Desktop$ sudo /var/ossec/bin/wazuh-logtest
[sudo: authenticate] Password:
Starting wazuh-logtest v4.14.7
Type one log per line

Aug 29 20:03:55 filterlog[52302]: 75,,,1787924727,en1,match,block,in,4,0x0,,64,33338,0,DF,1,icmp,84,192.168.10.100,142.251.153.4,request,10567,264

**Phase 1: Completed pre-decoding.
full event: 'Aug 29 20:03:55 filterlog[52302]: 75,,,1787924727,en1,match,block,in,4,0x0,,64,33338,0,DF,1,icmp,84,192.168.10.100,142.251.153.4,
request,10567,264'
timestamp: 'Aug 29 20:03:55'
hostname: 'filterlog[52302]:'

**Phase 2: Completed decoding.
name: 'pfsense-filterlog'
```

Figure 24: The second attempt, with the parent decoder matching by name but extracting no fields

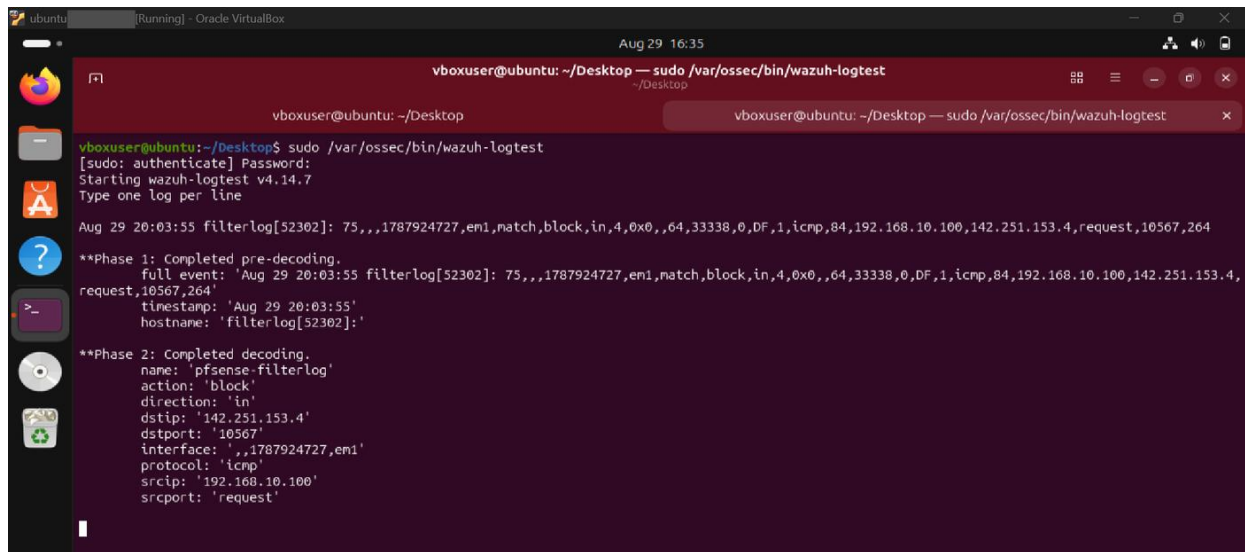


Figure 25: The fourth attempt, with the action, direction, protocol, addresses and ports all extracted into named fields

Why the Patterns Look Like That

The pattern syntax deserves attention, because Wazuh does not use the regular expression dialect most people are used to. In its default engine a plain full stop means a literal dot, `\.` means any character, and square-bracket character classes are not supported and silently match nothing. This is the same trap I recorded in the earlier project, where an entirely ordinary address pattern of `([0-9.]+)` had to become `(\S+)` before it would match anything at all.

Counting comma separated fields with `\S*`, and extracting with `(\S+)` sidesteps the question completely, and it is the idiom the platform's own shipped decoders use. Perl-compatible syntax is available where it is genuinely needed, by setting a type attribute on the expression, but it is easier to get quietly wrong, so I stayed with the simple engine.

The Decoder

```

<decoder name="pfsense-filterlog">
  <prematch>,match,</prematch>
</decoder>

<decoder name="pfsense-filterlog-ports">
  <parent>pfsense-filterlog</parent>

  <regex>,(\\S+),match,(\\S+),(\\S+),4,\\S*,\\S*,\\S*,\\S*,\\S*,\\S*,\\S*,(\\S*),\\S*,(\\S*), (\\S*), (\\S*), (\\S*),</regex>
  <order>interface,action,direction,protocol,srcip,dstip,srcport,dstport</order>
</decoder>

<decoder name="pfsense-filterlog-noports">
  <parent>pfsense-filterlog</parent>

  <regex>,(\\S+),match,(\\S+),(\\S+),4,\\S*,\\S*,\\S*,\\S*,\\S*,\\S*,\\S*,(\\S*),\\S*,(\\S*), (\\S*),</regex>
  <order>interface,action,direction,protocol,srcip,dstip</order>
</decoder>

```

Two child decoders are needed because not every message carries ports. Messages describing transmission control protocol or user datagram protocol traffic end with a source and destination port; messages describing internet control message protocol traffic end with a type and a sequence number instead. The variant expecting ports is listed first, because decoding stops at the first child that matches.

The file lives in `/var/ossec/etc/decoders/` and has to be readable by the analysis engine. A decoder file with the wrong ownership is ignored in silence, which from the validation tool is indistinguishable from a decoder that does not match.

12. Detection Rules and Correlation

A decoder answers what the log says, field by field, and makes no judgement about any of it. A rule answers whether what the log says matters, at what severity, and under what classification. Both are required, and they have to agree exactly about field names.

Three Rules

Identifier	Level	What it matches	Framework mapping
100900	5	Any decoded firewall message, reporting source, destination, protocol and direction	—
100901	3	Permitted traffic, at a lower severity because it is context rather than an incident	—
100902	10	Repeated matches of rule 100900 from the same source inside sixty seconds	T1595, Active Scanning

```
<group name="pfsense,pfsense_custom,">

  <rule id="100900" level="5">
    <decoded_as>pfsense-filterlog</decoded_as>
    <description>pfsense: packet blocked from $(srcip) to $(dstip) over
$(protocol).</description>
    <group>firewall_block,</group>
  </rule>

  <rule id="100901" level="3">
    <decoded_as>pfsense-filterlog</decoded_as>
    <description>pfsense: packet permitted from $(srcip) to $(dstip).</description>
    <group>firewall_pass,</group>
  </rule>

  <rule id="100902" level="10" frequency="8" timeframe="60">
    <if_matched_sid>100900</if_matched_sid>
    <same_source_ip />
    <description>pfsense: repeated blocked packets from $(srcip) within one
minute.</description>
    <mitre>
      <id>T1595</id>
    </mitre>
    <group>firewall_block,recon,</group>
  </rule>
```

```
</group>
```

Custom rule identifiers start at 100000 so that they never collide with the shipped ruleset, which numbers in the tens of thousands. I numbered these from 100900 to leave the rule written in the earlier project untouched at 100003. This work extends that environment rather than replacing it.

Rule 100902 is the one that earns its place. A single blocked packet is noise. Repeated blocked packets from one source inside a minute is a pattern, and describing the pattern once is more useful to an analyst than describing each packet eight times. `<if_matched_sid>` makes it fire on repetition of the first rule rather than on any single event, and `<same_source_ip />` groups the count by source so that unrelated activity from different machines does not accumulate into a pattern that was never there. It is mapped to Active Scanning under the Reconnaissance tactic, in the same way the earlier project's rule was mapped to Brute Force.

A Rule That Loaded and Would Not Fire

My first version tested the decoded action field explicitly, so that one rule caught denials and the other caught permitted traffic. The manager refused to start:

```
wazuh-analysisd: ERROR: Failure to read rule 100900. Field 'action' is static.  
wazuh-analysisd: CRITICAL: (1220): Error loading the rules
```

The name `action` is reserved as a static field in the rule language and cannot be used as a general field test. The alternative form of the condition loaded cleanly, and then never fired. In the end the condition was removed from both rules, which has a consequence recorded honestly in Section 14.

Two things came out of that.

The first is the value of `wazuh-analysisd -t`, which parses the entire ruleset and reports errors without touching the running service. Running it before a restart is the difference between a five-second correction and a manager that will not start at all, taking agent log processing down with it for a reason buried in a log file rather than shown on the screen.

The second is that a rule file which loads is not the same thing as a rule which fires. That distinction is why every change from this point on was validated against a captured line rather than by generating traffic and hunting the dashboard for a result.

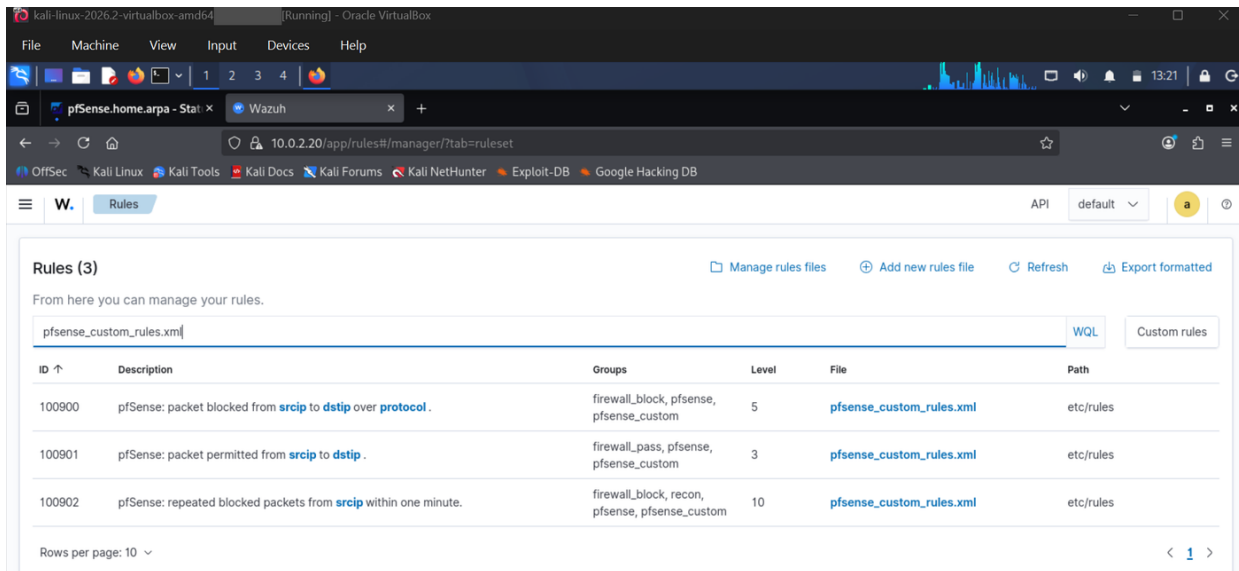


Figure 26: The three custom rules loaded by the manager, with their severity levels and groups

Validating Before Generating Traffic

```
sudo /var/ossec/bin/wazuh-analysisd -t
sudo systemctl restart wazuh-manager
sudo /var/ossec/bin/wazuh-logtest
```

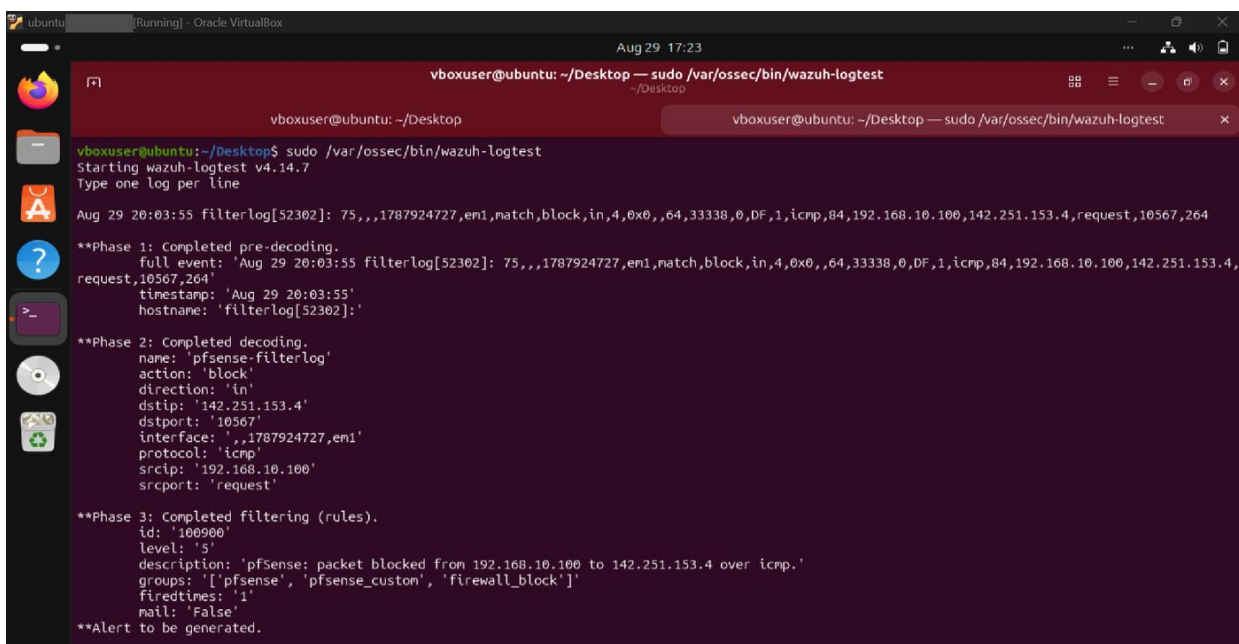


Figure 27: All three phases succeeding, with the custom decoder extracting fields and rule 100900 classifying the event at level 5

All three phases succeed. The message is decoded by the custom decoder, the extracted values are substituted into the alert description, and the rule identifier and level classify it. This is the same discipline that made the earlier project tractable, and it is the step I would tell anyone not to skip: a rule tested against a known line either works or fails visibly in two seconds, while a rule tested by generating traffic can fail for a dozen unrelated reasons and separating them takes an hour.

The Live Alerts

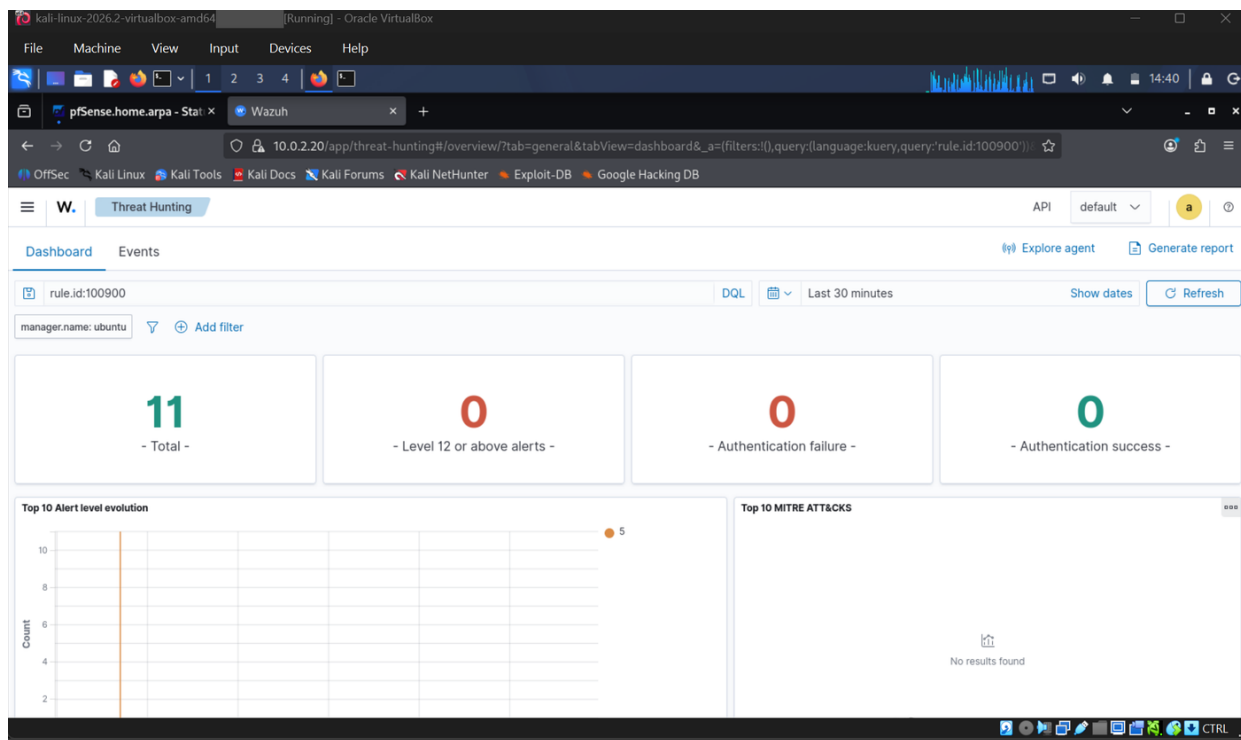


Figure 28: Blocked packets from the firewall arriving as alerts under custom rule 100900

There was one detour worth recording between the validation passing and this view appearing. Alerts were being written to the manager's own alert log while the dashboard showed nothing at all, which looks exactly like a broken detection. Two things contributed: a filter left pinned on the search from earlier work, and the firewall and the monitoring server sitting in different time zones, roughly five hours apart, so that a short dashboard time range excluded every event that had just been generated. Two machines that disagree about the time make correlation impossible, and the symptom does not point anywhere near the cause.

Where the Alert Says It Came From

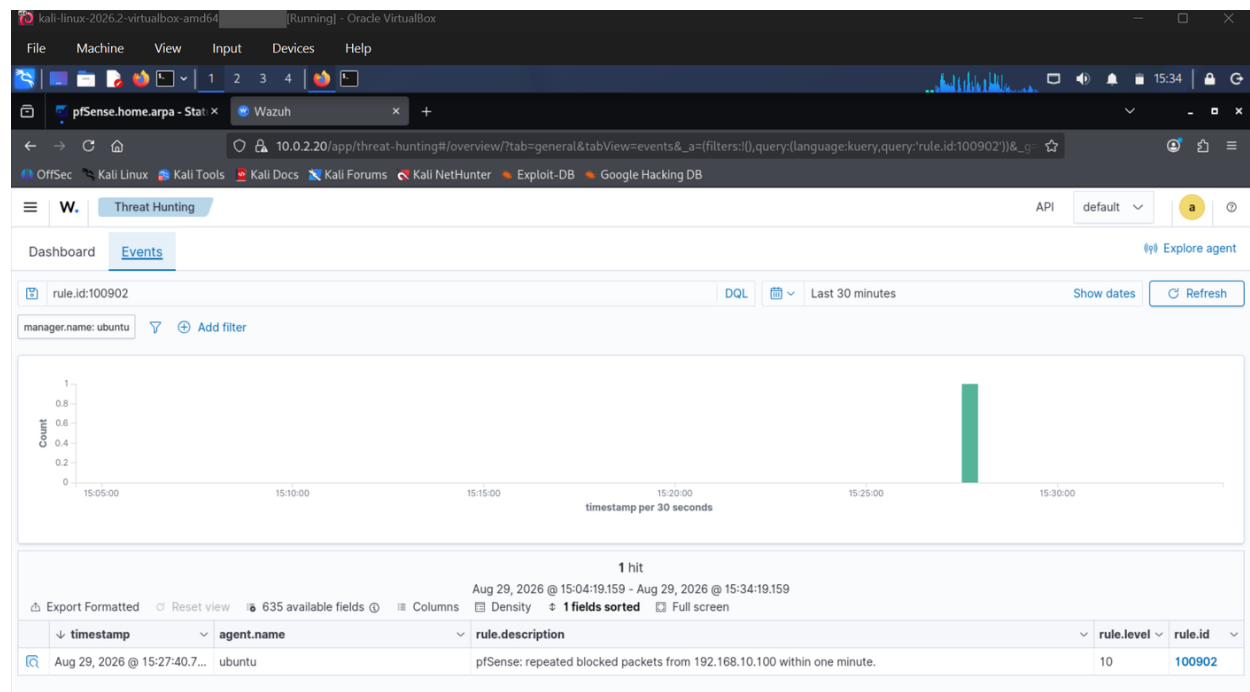


Figure 29: The correlation alert attributed to the manager rather than to the endpoint that generated the traffic

The agent column reads `ubuntu`. Not the endpoint, which generated every one of these packets, and not the firewall, which observed and dropped them.

The reason is the transport. These messages arrive over the system logging protocol with no agent behind them, so there is no agent identity to attribute them to and they are recorded against the manager itself. The sending device appears in the location field instead.

The practical consequence is worth more than the explanation. Filtering the dashboard by the endpoint's agent name returns nothing at all, for events the endpoint caused. An analyst who starts from the machine they suspect will conclude that the detection is broken. The filters that work are the rule group and the decoded source address, both of which are populated on every one of these alerts.

This is the same lesson as the earlier project in a new form: an alert belongs to whatever produced the log, not to whatever caused the activity.

Correlation

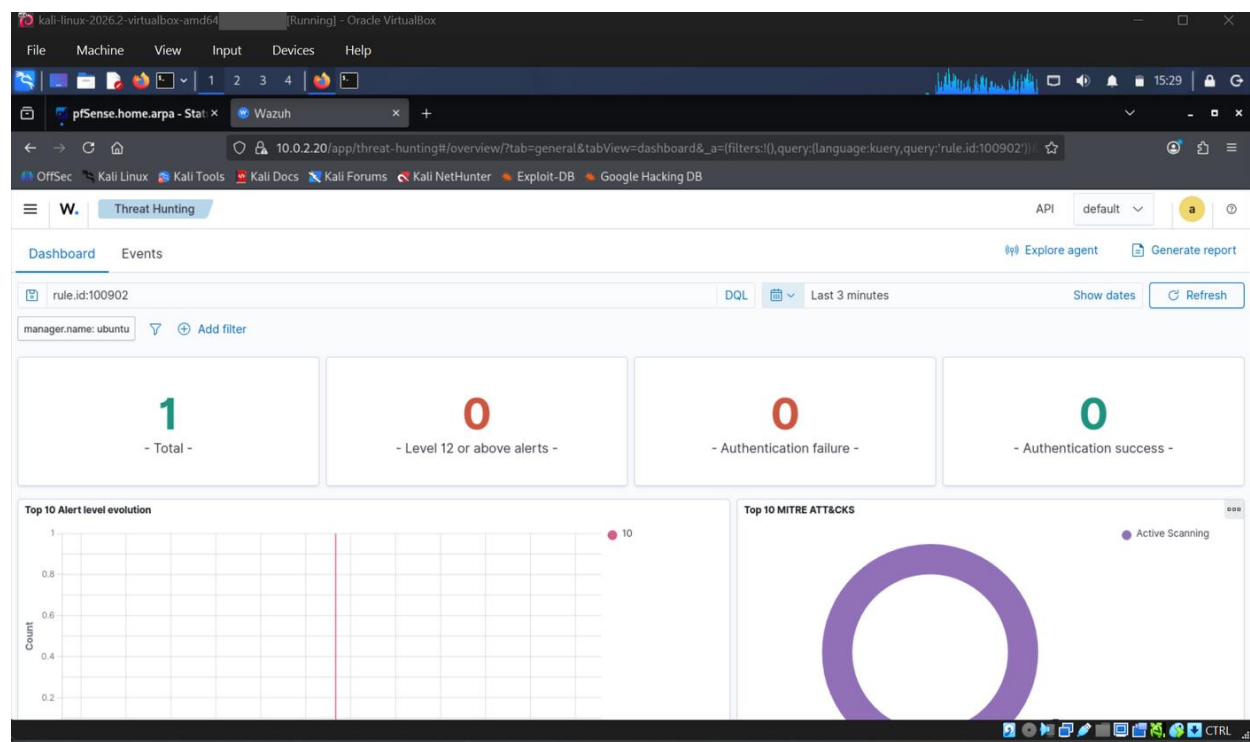


Figure 30: The correlation rule raising a single level 10 alert, carrying its Active Scanning mapping

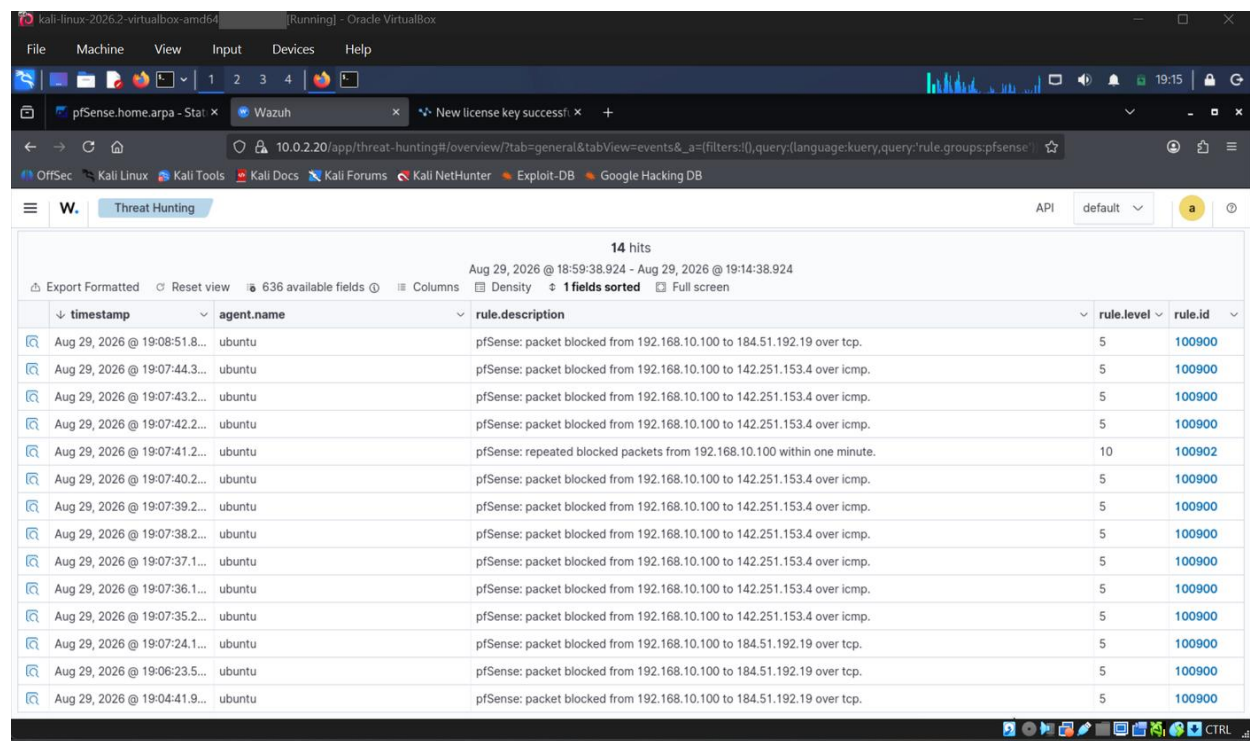


Figure 31: Individual blocked packets and the correlation alert that summarises them, in one view

The correlation rule fires, at level 10, with its framework mapping attached, and it appears alongside the individual events it summarises rather than instead of them, which is what makes both views useful at once.

I have deliberately not stated the exact number of packets that triggers it. The frequency counter in this platform has historically been off by one or two from the literal value written in the rule, and stating a threshold would be assertion rather than observation. What can be stated is what was observed: a burst of sixteen requests to a blocked address produced fourteen individual alerts and two correlation alerts inside the same window.

Tracing a Rule End to End

The last test was a rule written for no reason other than to watch something travel the full pipeline: a rule denying outbound internet control message protocol traffic from the endpoint, which is broad enough that a single command produces a clean and unambiguous burst.

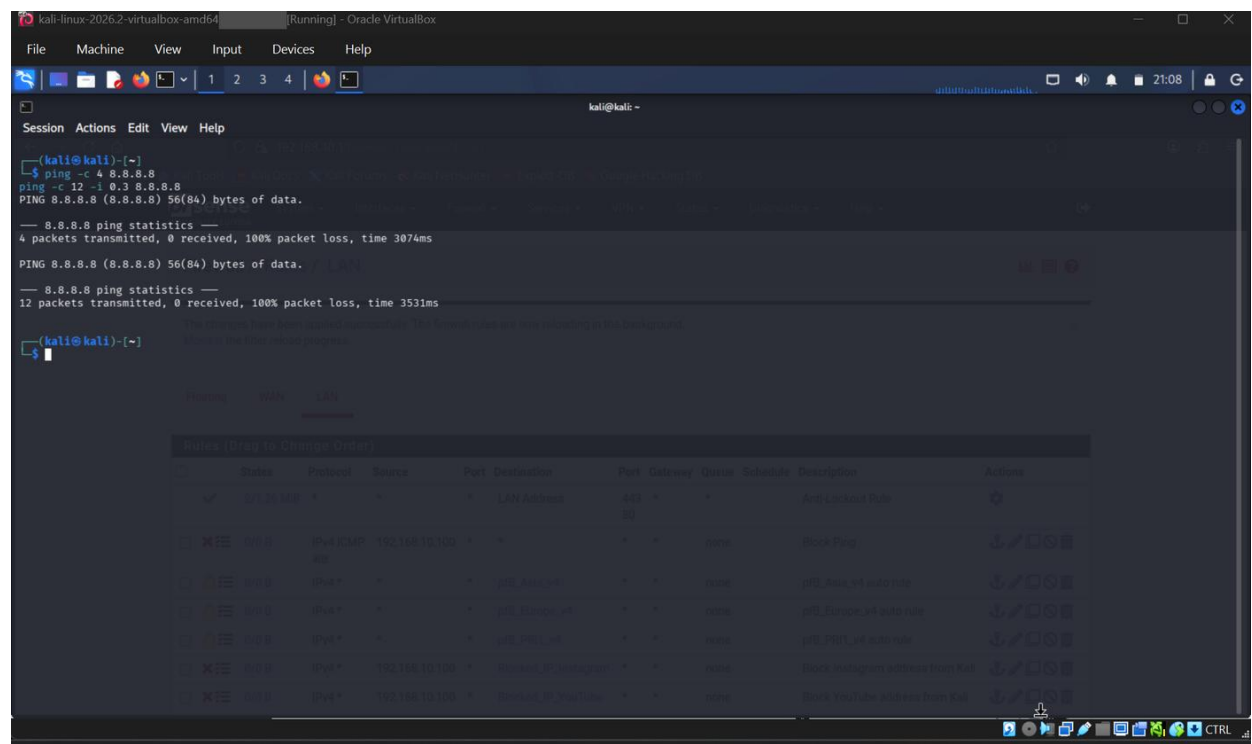


Figure 32: The new rule taking effect, with requests from the endpoint to a public address failing completely

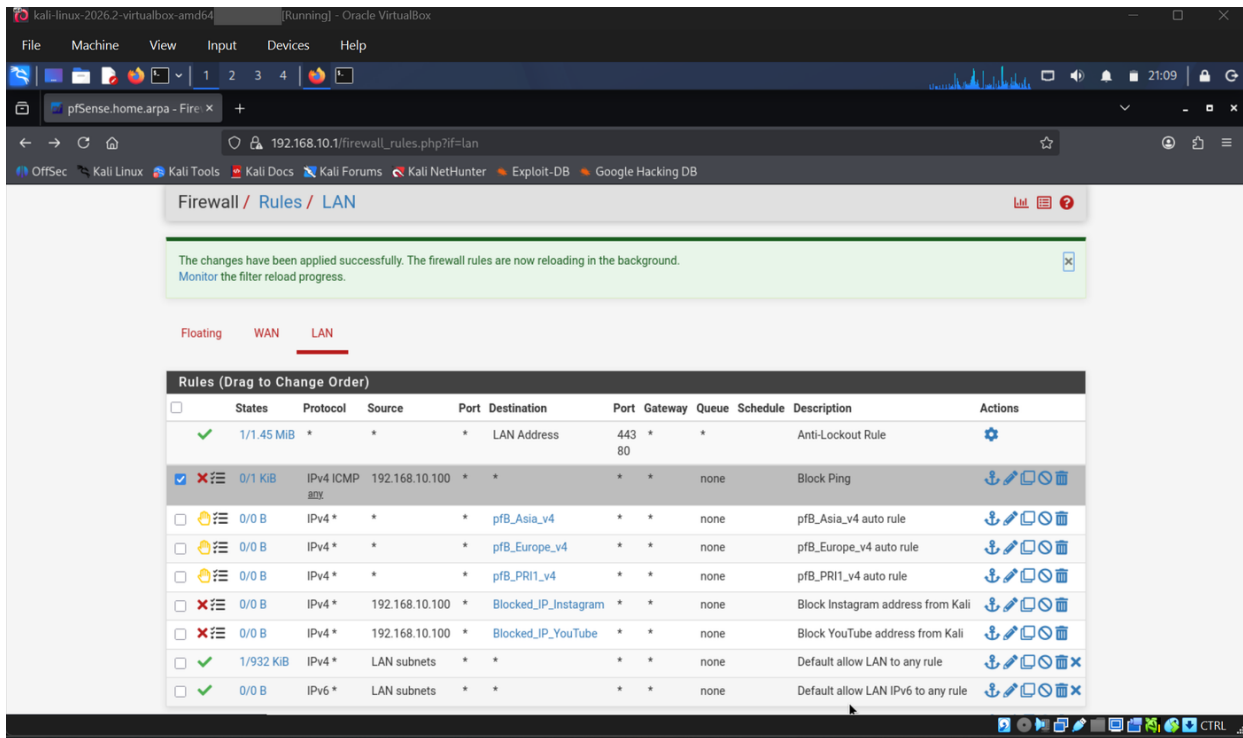


Figure 33: The full rule list on the inner interface, showing the hand-written rules, the new rule, and the rules generated by the filtering package

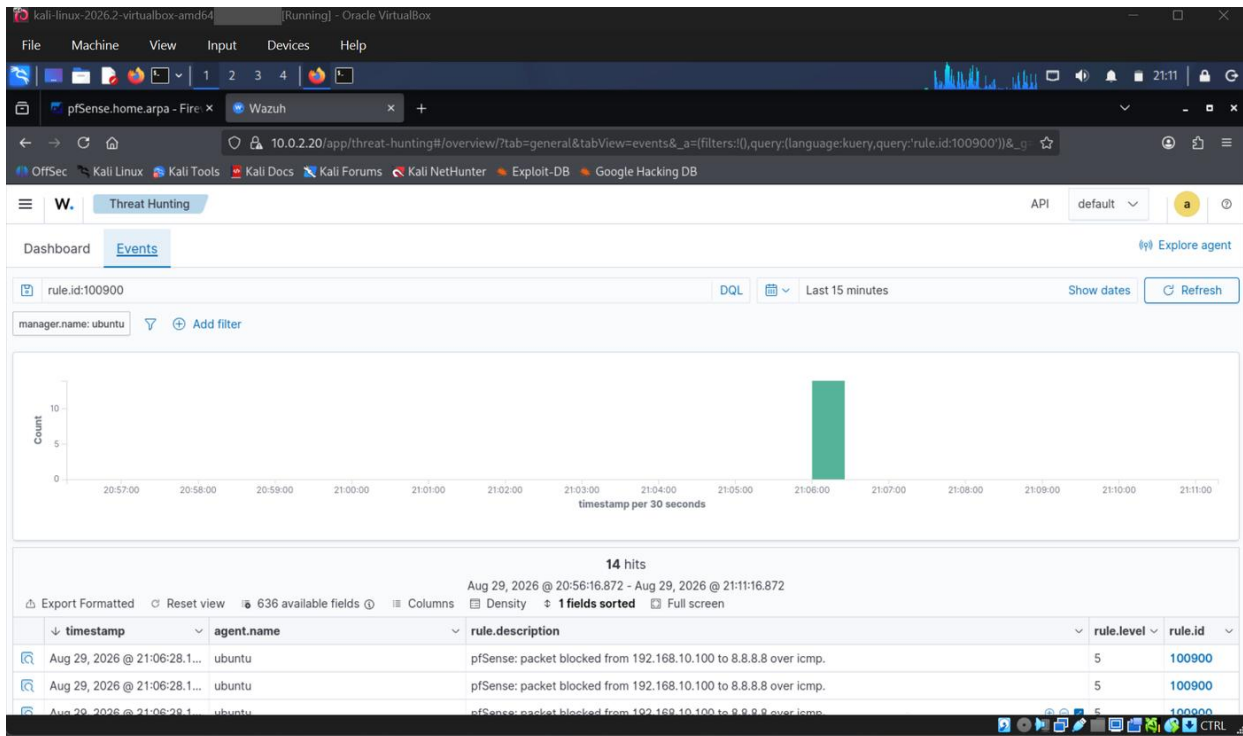


Figure 34: The denied requests arriving in the monitoring platform as individual alerts under rule 100900

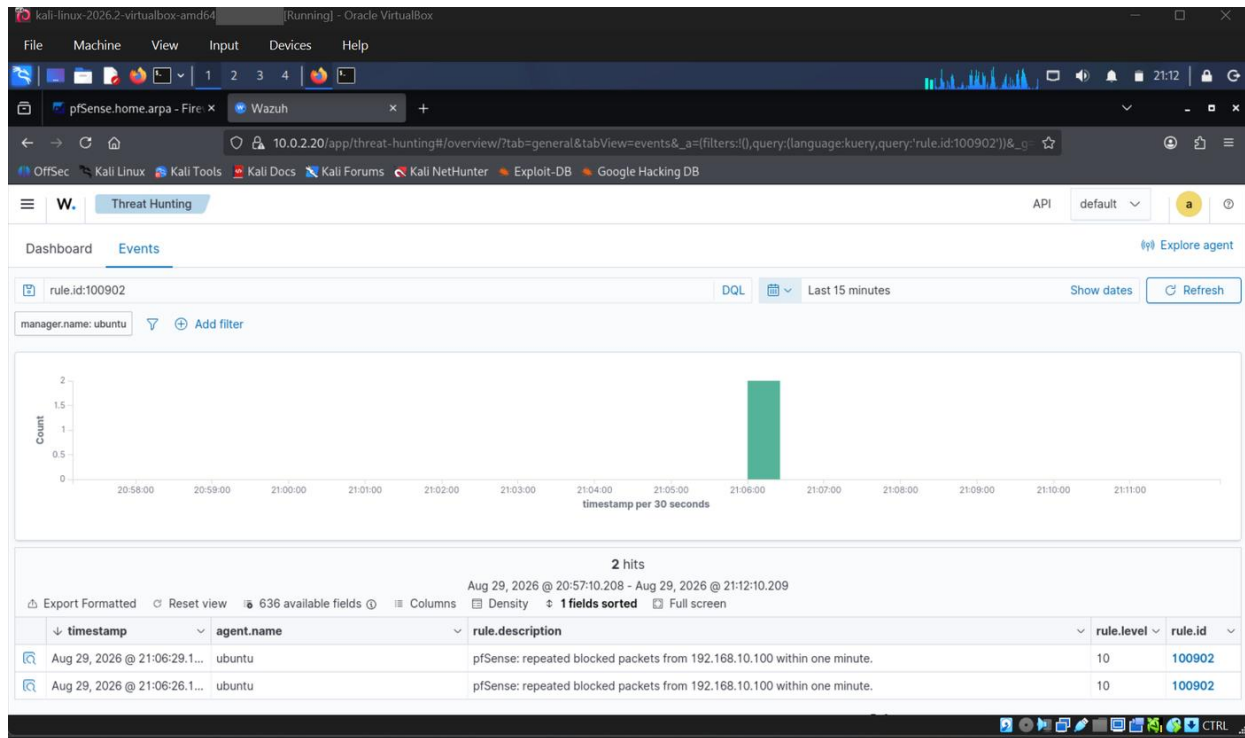


Figure 35: The correlation rule firing twice across the same burst

Every stage is visible: the rule matching at the firewall, the traffic failing at the endpoint, the individual alerts in the platform, and the correlation alert summarising them. That progression from a dropped packet to a classified, queryable security event carrying a framework mapping is the entire argument for putting a monitoring platform behind a firewall rather than reading the firewall's own log.

Part IV: Analysis and Reflection

13. Visualizing the Firewall Activity

A table of alerts answers what happened. A dashboard answers whether the pattern is changing, which is the question an analyst actually asks during a shift.

Saved Search

A saved search stores a filter as a reusable object, so that anything built on top of it stays tied to the detection rather than to a query somebody has to remember.

```
Name      pfSense Firewall Blocks
Filter     rule.groups:pfsense
Columns    timestamp, rule.id, data.srcip, data.dstip, data.protocol, rule.description
```

Visualizations

Two, both vertical bar charts, answering different questions about the same data.

- **Firewall Block Trend** counts events over a date histogram. It answers when.
- **Blocked Destinations by Volume** counts events grouped by destination address. It answers where.

```
Firewall Block Trend
vertical axis    Count
horizontal axis  date histogram on timestamp, automatic interval

Blocked Destinations by Volume
vertical axis    Count
horizontal axis  terms aggregation on data.dstip, size 10, descending
```

The Dashboard

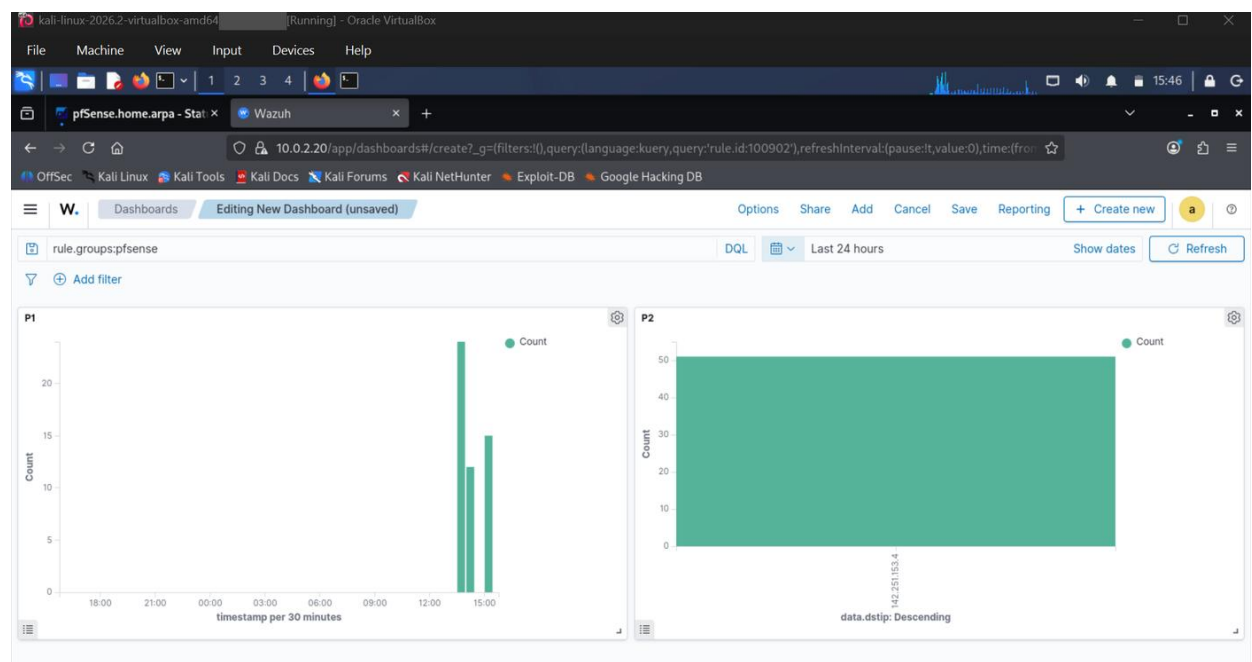


Figure 36: The two visualisations combined on one dashboard, showing the shape of the activity over time and the destinations attracting it

Combining the trend with the destination breakdown gives shape and detail in a single view. This deliberately follows the same pattern as the dashboard built in the earlier project, so that the two are directly comparable rather than each being its own invention.

14. Verification Summary and Limits of the Design

What Was Verified

Every capability claimed in this document appears below with the test that was run and what it returned, including the two that did not hold.

Capability	How it was tested	Evidence	Result
The endpoint sits behind the firewall with no path around it	Address, route and name server read from the endpoint after the move	Figure 7	Confirmed
The earlier monitoring build survives the move	Agent status queried on the manager; dashboard opened from the endpoint	Figures 8 and 10	Confirmed
Named service addresses are blocked from the endpoint	Both addresses attempted, with two unrelated destinations as controls	Figure 14	Confirmed
Denials are recorded by the firewall against the rule that caused them	Firewall log inspected after the test	Figure 15	Confirmed

Capability	How it was tested	Evidence	Result
The rules are matching rather than merely present	Byte counters read on both rules	Figure 16	Confirmed
Firewall messages reach the manager	Traffic captured on the manager's interface on port 514	Figure 22	Confirmed
Messages are decoded into named fields	Captured line replayed through the validation tool	Figure 25	Confirmed
Blocked packets raise classified alerts	Validation tool, then live traffic observed in the dashboard	Figures 27, 28 and 34	Confirmed
Repetition is correlated into one higher severity alert	Burst of sixteen requests from a single source	Figures 30, 31 and 35	Confirmed
Geographic filtering denies the selected countries	Alias tables built and generated rules inspected	Figure 19	Partial. The lists were built and the rules generated, but the block itself was not separately verified from the endpoint
Permitted traffic raises a distinguishable alert	Searched for rule 100901 after generating permitted traffic	—	Not achieved. See below
Domain filtering sinkholes the listed domains	Name lookups from the endpoint	Figure 20	Not achieved. See Section 8

Where the Design Cannot See

- **Only what crosses the firewall.** Traffic between two machines on the same protected segment never reaches the firewall and therefore never appears in any of these detections. The field of view is the boundary, not the network.
- **Only what the firewall is configured to log.** A rule without logging enabled protects the network and tells the platform nothing. Permitted traffic is largely invisible here by choice, for the reason given in Section 9.
- **Encrypted name resolution defeats domain filtering entirely.** A browser can resolve names over its own encrypted connection to a public resolver, bypassing the firewall's resolver completely. A domain can be sinkholed correctly at the command line and still load in that browser. Domain filtering at a perimeter is therefore a control that works on cooperative clients rather than a boundary that holds, and any honest account of it has to say so.
- **Blocking one address does not deny a service.** Both services blocked in Section 6 answer from pools of thousands of addresses. The rules demonstrate the mechanism and deny that particular address; they do not deny the service, and nothing in this build claims otherwise.

Known Imperfections in the Delivered Build

- **The interface field decodes incorrectly**, as `, , 1787924727, em1` rather than `em1`, because the child pattern is one field out. Every field the rules actually use, which is source, destination, action, direction and protocol, decodes correctly.
- **For messages without ports, the source port decodes as `request`**, because the variant expecting ports matches internet control message protocol lines that it should not. The correct fix is a tighter pre-match on the child rather than the ordering trick used here.
- **Rule 100901 is effectively dead**. With the action condition removed from both, rules 100900 and 100901 have identical matching conditions, and the more specific of two equal rules wins consistently. Permitted traffic is not separately alerted on. Fixing it needs a discriminator that is not the reserved field name, which is the first thing I would return to.
- **The exact correlation threshold is unmeasured**. What was observed is recorded in Section 12; the literal value in the rule is not claimed as the firing point.

15. Problems Encountered

Each failure is documented where it happened, with the reasoning that identified it. This table is the index rather than a second telling.

Symptom	Cause	Fix	Where shown
The endpoint receives no address after being moved	Stale fixed name server configuration left from earlier work	Cleared it and requested configuration from the firewall again	Section 5
The firewall reaches everything from its own console, but nothing behind it reaches anything	An address was set, but no upstream gateway was selected. A route is a separate object from an address	Selected the gateway on the outer interface	Section 4
A rule reads correctly, the traffic passes, and the counters stay at zero	The rule is on the interface the traffic does not enter	Placed both rules on the interface where the endpoint's traffic enters the firewall	Sections 3 and 6
Messages arrive at the manager and no alerts appear anywhere	Message format set to the newer structured standard, which the pre-decoding stage does not expect	Set the format back to the older standard	Section 9
No <code>decoder matched</code> for a correctly formatted message	The firewall sends no hostname, which displaces the program name the shipped decoder identifies on	Wrote a decoder that identifies the message by its structure instead	Sections 10 and 11
The decoder matches but extracts no fields	Child patterns counted fields from the start of a string the pre-decoder had already consumed	Re-anchored the child patterns on the literal that identifies the message	Section 11

Symptom	Cause	Fix	Where shown
The manager refuses to start after a rule change	<code>action</code> is a reserved static field and cannot be used as a general field test	Tested the ruleset offline before restarting, and removed the condition	Section 12
Validation passes and alerts are written, but the dashboard is empty	A filter left pinned on the search, and two machines five hours apart in their time zones	Cleared the filter and set both machines to the same time zone	Section 12

16. Key Takeaways

- **A missing field can defeat a working decoder.** Nothing has to be misconfigured for an integration to fail completely. Two products can each be entirely correct and still disagree about whether an optional field is optional, and the only way to see it is to read the phase output rather than to guess.
- **Identify a message by what is present, not by what should be.** A decoder anchored on the structure of a message survives the sender's choices about optional fields. One anchored on a single field does not.
- **Rules are evaluated where packets enter.** Nearly every instance of "the rule looks right and does nothing" in a firewall comes back to this, and the byte counter is the diagnostic that separates it from everything else.
- **An address is not a route.** The test that proves an address is reachable proves nothing about whether there is a path onward, and from behind the firewall the two failures are indistinguishable.
- **Validate offline before generating traffic.** Adopted partway through the earlier project and carried into this one from the start. It is the single largest difference in the pace of the work.
- **An alert belongs to whatever produced the log.** Filtering by the machine you suspect is how a working detection gets reported as broken.
- **Correlation is what the monitoring platform adds.** A firewall log records fifteen denials. A correlation rule says one sentence about them, and that sentence is the thing an analyst can act on.
- **Name what you mean at a boundary.** Selecting a region rather than four countries would have denied the country the laboratory runs in, and the cause would have been several menus from where anyone would look.
- **Report what did not work.** The domain filter and the dead permit rule are in this document because a build described only by its successes is not a description of a build.

What I Would Do Differently

- **Give the permitted-traffic rule a real discriminator.** Rule 100901 is dead because the obvious field name is reserved. Matching on the decoded direction, or on a tighter child decoder that only claims denials, would separate the two cases properly.

- **Measure the correlation threshold rather than describing it as unmeasured.** A controlled burst of a known size, repeated, would turn an honest gap into a stated fact.
- **Fix the two decoder field offsets.** Both are cosmetic today and both would matter the moment somebody searched on the interface field.
- **Add a network sensor.** The firewall reports its own verdicts. A sensor watching the traffic itself would report what the traffic looks like, which is a different and complementary thing, and it is the natural next extension of this laboratory.

Appendix A: Command Reference

The commands below are the ones that mattered, in the order they were run.

On the monitoring server

```
# system log listener
sudo cp /var/ossec/etc/ossec.conf /var/ossec/etc/ossec.conf.bak-$(date +%F)
sudo nano /var/ossec/etc/ossec.conf
sudo systemctl restart wazuh-manager
sudo ss -ulnp | grep :514

# confirm messages are arriving
sudo tcpdump -n -i enp0s3 -A udp port 514

# decoder and rules
sudo nano /var/ossec/etc/decoders/pfsense_custom_decoders.xml
sudo nano /var/ossec/etc/rules/pfsense_custom_rules.xml
sudo chown wazuh:wazuh /var/ossec/etc/decoders/pfsense_custom_decoders.xml
sudo chown wazuh:wazuh /var/ossec/etc/rules/pfsense_custom_rules.xml
sudo chmod 660 /var/ossec/etc/rules/pfsense_custom_rules.xml

# validate, then restart, then validate again
sudo /var/ossec/bin/wazuh-analysisd -t
sudo systemctl restart wazuh-manager
sudo /var/ossec/bin/wazuh-logtest

# agents and time
sudo /var/ossec/bin/agent_control -l
sudo timedatectl set-timezone Asia/Karachi
```

On the monitored endpoint

```
# position after the move
ip -brief address show
ip route
cat /etc/resolv.conf
sudo dhclient -r eth0 && sudo dhclient eth0

# reachability
ping -c 3 192.168.10.1
ping -c 3 10.0.2.20
nc -zvn 10.0.2.20 1514
sudo systemctl restart wazuh-agent
```

```
# rule verification, with controls
ping -c 4 142.251.153.4
ping -c 4 157.240.227.174
ping -c 3 8.8.8.8
curl -I --connect-timeout 5 http://example.com

# correlation burst
ping -c 16 -i 0.3 142.251.153.4

# name resolution checks
nslookup youtube.com
nslookup instagram.com
nslookup example.com
```

On the firewall console

Option	Use
1	Assign interfaces
2	Set interface addresses
5	Reboot
7	Ping host, the quickest connectivity test available on the device
8	Shell
11	Restart the web interface if it becomes unreachable

Appendix B: Decoder and Rule Files

/var/ossec/etc/decoders/pfsense_custom_decoders.xml

```
<!-- Identifies pfSense firewall messages by their structure rather than by
      program name, because this firewall sends no hostname and the program
      name is therefore displaced out of the field the pre-decoder reads. -->

<decoder name="pfsense-filterlog">
  <prematch>,match,</prematch>
</decoder>

<!-- Messages carrying ports. This variant must appear first, because
      decoding stops at the first child that matches. -->
<decoder name="pfsense-filterlog-ports">
  <parent>pfsense-filterlog</parent>

  <regex>,(\\S+),match,(\\S+),(\\S+),4,\\S*,\\S*,\\S*,\\S*,\\S*,\\S*,\\S*,(\\S*),\\S*,(\\S*), (\\S*), (\\S*), (\\S*),</regex>
  <order>interface,action,direction,protocol,srcip,dstip,srcport,dstport</order>
</decoder>

<!-- Messages without ports, such as those produced by the internet control
      message protocol, which end with a type and a sequence number. -->
<decoder name="pfsense-filterlog-noports">
  <parent>pfsense-filterlog</parent>
```

```
<regex>,(\\S+),match,(\\S+),(\\S+),4,\\S*,\\S*,\\S*,\\S*,\\S*,\\S*,\\S*,(\\S*),\\S*,(\\S*), (\\S*),<
/regex>
  <order>interface,action,direction,protocol,srcip,dstip</order>
</decoder>
```

[/var/ossec/etc/rules/pfsense_custom_rules.xml](#)

```
<!-- Firewall detection rules. Numbered from 100900 to leave rule 100003,
      written for the earlier application log detection, untouched. -->
<group name="pfsense,pfsense_custom,">

  <rule id="100900" level="5">
    <decoded_as>pfsense-filterlog</decoded_as>
    <description>pfSense: packet blocked from $(srcip) to $(dstip) over
$(protocol).</description>
    <group>firewall_block,</group>
  </rule>

  <rule id="100901" level="3">
    <decoded_as>pfsense-filterlog</decoded_as>
    <description>pfSense: packet permitted from $(srcip) to $(dstip).</description>
    <group>firewall_pass,</group>
  </rule>

  <rule id="100902" level="10" frequency="8" timeframe="60">
    <if_matched_sid>100900</if_matched_sid>
    <same_source_ip />
    <description>pfSense: repeated blocked packets from $(srcip) within one
minute.</description>
    <mitre>
      <id>T1595</id>
    </mitre>
    <group>firewall_block,recon,</group>
  </rule>

</group>
```

Appendix C: Sample Messages

A denial carrying no ports, as received

```
Aug 29 20:03:55 filterlog[52302]: 75,,,1787924727,em1,match,block,in,4,0x0,,64,
33338,0,DF,1,icmp,84,192.168.10.100,142.251.153.4,request,10567,264
```

The absence of a hostname between the timestamp and `filterlog[52302]` : is the whole subject of Section 10. The line is shown wrapped here for the page; it arrives as a single line.

The same message decoded

```
**Phase 2: Completed decoding.  
  name: 'pfsense-filterlog'  
  action: 'block'  
  direction: 'in'  
  dstip: '142.251.153.4'  
  dstport: '10567'  
  interface: ',,,1787924727,em1'  
  protocol: 'icmp'  
  srcip: '192.168.10.100'  
  srcport: 'request'
```

The interface and srcport values are the two known imperfections recorded in Section 14. Every field the rules act on is correct.

A denial carrying ports, decoded

```
**Phase 2: Completed decoding.  
  name: 'pfsense-filterlog'  
  action: 'block'  
  direction: 'in'  
  dstip: '184.51.192.19'  
  dstport: '80'  
  protocol: 'tcp'  
  srcip: '192.168.10.100'  
  srcport: '58556'  
  
**Phase 3: Completed filtering (rules).  
  id: '100900'  
  level: '5'  
  description: 'pfSense: packet blocked from 192.168.10.100  
               to 184.51.192.19 over tcp.'  
  groups: '['pfsense', 'pfsense_custom', 'firewall_block']'  
  
**Alert to be generated.
```